ASMPT

TU/e EINDHOVEN UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

Model Augmentation for a High-Precision Dual-Drive Gantry

by

D.J.L.M. van den Berg

MSc Thesis

Thesis committee

dr. ir. M. Schoukens
prof. dr. ir. R. Tóth
dr. ir. M.F. Shakib
ir. J. Gerritsen

Graduation

Program:	Artificial Intelligence and Engineering Systems
Research group:	Control Systems
Thesis supervisor:	dr. ir. M. Schoukens
Date of defense:	17-11-2026
Student ID:	1555561
Study load (ECTS):	45
Track:	High-Tech Systems and Robotics

The research of this thesis has been carried out in collaboration with ASMPT, Beuningen.

This thesis is public and Open Access.

This thesis has been realized in accordance with the regulations as stated in the TU/e Code of Scientific Conduct.

Disclaimer: the Department of Electrical Engineering of the Eindhoven University of Technology accepts no responsibility for the contents of MSc theses or practical training reports.

Model Augmentation for a High-Precision Dual-Drive Gantry

Dirk van den Berg
Control Systems Group, Dept. of Electrical Engineering
Eindhoven University of Technology
Eindhoven, The Netherlands
In collaboration with *ASMPT*, Beuningen, The Netherlands

Supervisors: dr. ir. M. Schoukens, prof. dr. ir. R. Tóth, ir. J.H. Hoekstra (TU/e);
ir. Q. van den Elsen, ir. J. Gerritsen (ASMPT)

***Abstract*—**

- **Problem: improve settling prediction for a position-dependent dual gantry without surrendering the meaning of its physical parameters**
- **Method: combine a quasi-LPV first-principles baseline, dynamic parallel LFR augmentation, closed-loop rollout training, and state-level orthogonal projection**
- **Evidence: compare baseline, unconstrained augmentation, orthogonal variants, and a black-box model on the completed validation ladder, including relevant motion and settling where available**
- **Main result: report predictive improvement together with the measured limits from short-horizon training, inactive added states, and non-orthogonal true missing dynamics**
- **Conclusion: state when the proposed decomposition is useful and which conditions prevent interpreting it as true parameter recovery**

[TODO: Write the numerical abstract only after the final simulation and Telica result tables are frozen.]

***Index Terms*—**

I. Introduction

This thesis focuses on a dual-drive gantry motion system in which two parallel linear actuators support a common beam carrying the payload. The gantry positions this payload at successive placement locations. The shared beam couples the dynamics of both actuators, so the motion of one actuator influences the other, and the system exhibits position-dependent behaviour [1], [2] and flexible dynamics [1]. This position dependence can be described by a linear parameter-varying (LPV) model. Furthermore, cross-coupling means that the motion of one actuator can disturb the other, which affects the tracking behaviour during simultaneous motion.

A plant model can be used to evaluate a changed feedback controller before it is tested on the machine. A closed-loop simulation is faster than an experiment, allowing multiple controller to be compared simultaneously, and without risk of harming the hardware. **[TODO:** Link to evaluation with a changed controller in closed loop (Section VI), once that result exists.] Such models are also valuable for root cause analysis. This requires a model with physically meaningful parameters, because a change in machine behaviour can then be traced back to a physical cause, such as a change in stiffness or damping. **[TODO:** Missing: source for the root cause analysis sentence. Candidate: Györök et al. Sec. 3.2 (non-unique estimate loses physical meaning) or Kessels Sec. 6.2 (not yet checked).] Both uses require an accurate representation of the underlying plant dynamics.

White-box models are attractive because they reflect the physics of the system, which makes them interpretable. First-principles (FP) models are a specific type of white-box model, derived from physical laws and described by quantities such as mass, stiffness, and damping. However, purely physics-based models do not always capture all behaviour needed for accurate prediction across the operating range [3]. More accurate FP models can be developed, but this is labour-intensive [4].

Purely data-driven (black-box) models such as neural-network or Gaussian-process state-space models can describe measured behaviour well, but can offer limited physical insight and are not guaranteed to be accurate at operating conditions not covered by the training data [3], [4]. These models can also require large amounts of data to train [3], [5]. In addition, these models need to learn behaviour that is already known from first principles, such as the rigid-body dynamics [4].

Physics-informed approaches, including physics-informed neural networks (PINNs), incorporate physical laws as constraints during training [5], [6]. However, these methods require the physics equations to be known and still rely on a black-box model for the entire system [4]. For the plant model of the dual-drive gantry, both predictive accuracy and physical interpretability are required. Physical interpretability means that Physical interpretability means that the identifiable combinations of the physical parameters ~~(Section III-C)~~[M-mail] are estimated uniquely [7]. This does not guarantee identifying the true parameters, but a deviation from the trained model can then indicate which physical parameter has changed. However, purely physics-based and purely data-driven models can each have limitations in meeting both requirements [3].

Grey-box modelling is the logical choice, because it is able to combine a baseline model with interpretable parameters with data-driven flexibility needed for accurate prediction [8]. Grey-box approaches include parameter re-estimation within the physics model structure, static residual correction on the model

output, and dynamic augmentation that extends the baseline with learned states [3], [4]. Model augmentation combines a physics-based baseline model with a learned component, so that the learned component only needs to capture the dynamics the physics-based model cannot describe [4]. Dynamic augmentation is chosen to be able to model missing dynamics, such as flexible dynamics. These require additional states, which parameter re-estimation and static output correction cannot provide [4]. However, related work at ASMPT on a wire bonder shows that preserving physical interpretability during model updating remains an open challenge [9]. Refitting the physical parameters allows them to absorbs unmodelled dynamics, and an unconstrained learning component is able to absorb dynamics the baseline already describes [7]. [**TODO:** Missing: check that Györök et al. supports the absorption by an unconstrained learned component, and a source for absorption by refitting. Candidate: Györök et al. Sec. 1 and Sec. 3.2 (not yet checked).]
Learning-based hybrid modelling has already been demonstrated using measurements from an industrial robot [10]. However, a key gap remains: there is no grey-box modelling framework for the dual-drive gantry that simultaneously addresses coupled multi-input multi-output (MIMO) dynamics, position-dependent behaviour, closed-loop data, and preserves physical interpretability. Model augmentation using a linear fractional representation (LFR) has already been demonstrated, but not for position-dependent baselines or closed-loop data [4]. The related work at ASMPT uses closed-loop data, but keeps the physical parameters fixed during training [9]. Orthogonal-by-construction (OBC) augmentation keeps the learned component orthogonal to the baseline parameters, so that their estimates are unique [7]. This has currently only been formulated for discrete-time input-output models, and not for state-space models. This thesis therefore develops such a framework and evaluates it in simulation. The augmented model should predict more accurately than the physics-based baseline across operating points and motion profiles, while its physical parameters remain interpretable.

This thesis addresses the following research question: *For the dual-drive gantry, how should a physics-based plant model be augmented to capture unmodelled dynamics, improving prediction accuracy across operating points and motion profiles while preserving the physical interpretability of its parameters?* To answer this research question, the following three aspects are investigated:

1) Which augmentation structure and training let the learned component capture the missing dynamics from closed-loop data?
2) To what extent does orthogonal-by-construction augmentation keep the physical parameters interpretable, compared to unconstrained estimation?
3) How does the augmented model generalise to held-out operating points and unseen motion profiles, compared to a black-box model?

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

The paper is organised as follows: first, the identification problem is stated in Section II, followed by the physics baseline in Section III. Then, Section IV introduces the augmented model and its identification in closed loop, and Section V the orthogonal construction. The experiment design is described in Section VI, and the results are given and discussed in Section VII. The conclusions are given in Section VIII. M-p3 [**Note:** Aspect 1 (LPV versus LTI) is removed: the LPV form follows from the equations of motion (Section III-A), so it is a modelling choice rather than a research result. The LTI comparison (on X the LTI error is 1.5 to 3.7 times the LPV error) moves to the appendix. Aspects 2 to 4 are reworded to the final method.] [**Note:** The current introduction is quite long, also the rq with aspects is quite long. Shall I remove the aspects, or shorten the introduction?]

[**TODO:** Confirm whether references and appendices are excluded from the 12-page IEEE limit and whether TU/e requires front matter beyond the IEEE paper.]

II. Identification Problem

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

The dual-drive gantry that we study in this thesis is a high-precision motion system of ASMPT, used in back-end semiconductor assembly and packaging equipment. The gantry has two actuators that are connected with a cross-arm and a third actuator that moves a payload along this arm (Fig. 1). [**TODO:** Js21 ("and presumably it has guidings and stages too, right?") and Q17 ("add location of encoders") open: the hardware of the ASMPT machine is confidential, and Garcia's description is of a different machine (Dirk 2026-10-18). Answer the reviewers directly.] The actuator forces are defined as $u_{\text{act}} = [F_{X1}\ F_{X2}\ F_Y]^\top$ and the measured positions $q_{\text{act}} = [X_1\ X_2\ Y]^\top$. Here F_{X1}, F_{X2} and F_Y are the forces and X_1, X_2 and Y the measured positions of the two X actuators

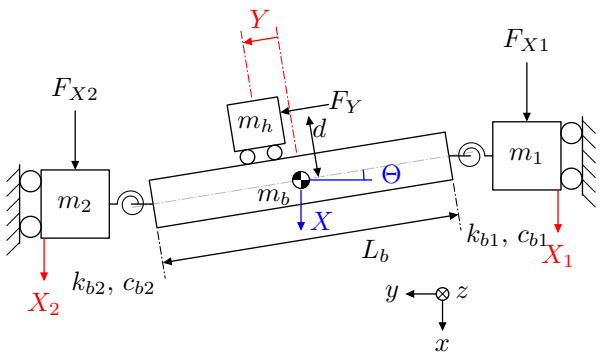


Fig. 1: Lumped-parameter model of the Dual-drive gantry stage, adapted from [1]. Red labels denote actuator coordinates and blue labels generalised coordinates.

[**TODO:** Will need to add springs to the current torsion; feedback from Jasper]

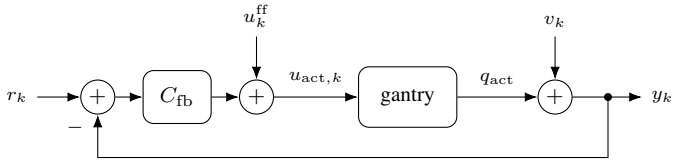


Fig. 2: Closed loop that generates the data: the controller C_{fb} of (2), the feedforward u_k^{ff}, the gantry (1) and the measurement noise v_k.

and the Y actuator.

The system to be identified is the gantry system described ~~as,~~ by

$$\dot{x}(t) = f\big(x(t), u_{\text{act}}(t)\big), \qquad y_k = q_{\text{act}}(kT_s) + v_k, \quad (1)$$

where the state x is unknown, except that it contains the actuator positions q_{act}, y_k are the measured positions at sample k with sampling time T_s, and v_k is zero-mean, coloured measurement noise. The state-transition function f is unknown. The system runs in closed loop with an assumed known linear time-invariant feedback controller ~~(Section IV-B)~~[M-p3], which gives the data set $\mathcal{D}_N = \{u_{\text{act},k}, y_k\}_{k=1}^N$. The controller computes the actuator forces from the position error $r_k - y_k$ between the reference r_k and the measured positions, and a feedforward u_k^{ff} is added (Fig. 2):[M-p3]

$$\begin{aligned} x_{k+1}^{\text{fb}} &= A_{\text{fb}} x_k^{\text{fb}} + B_{\text{fb}}(r_k - y_k), \\ u_{\text{act},k} &= C_{\text{fb}} x_k^{\text{fb}} + D_{\text{fb}}(r_k - y_k) + u_k^{\text{ff}}, \end{aligned} \quad (2)$$

[from IV-B, M-p3] where $(A_{\text{fb}}, B_{\text{fb}}, C_{\text{fb}}, D_{\text{fb}})$ are the state-space matrices of the controller, with state x^{fb},[M-p3] discretised with sampling time T_s. Both r_k and u_k^{ff} are in actuator coordinates.[end] The noise v_k is independent of r and u^{ff}.[M-p3]

A baseline model of (1) with physical parameters is available ~~(Section III-A)~~[M-mail]. [from III intro, Jan7] The dynamics of the system that are not captured by the baseline model ~~defined in Section III-B~~ will be identified from ~~input-output data measured in closed-loop~~ $\mathcal{D}_N$[Jan7]. The baseline model will be augmented to represent these missing dynamics.[end] A correction to the states of the baseline model alone is not sufficient for missing dynamics such as a flexible mode, because these need states the baseline does not have, so additional states are introduced. We make use of a feedforward neural network to model both the corrections to the baseline states and the additional states.

The goal is to estimate from $\mathcal{D}_N$ a model that combines this baseline with learned components and captures the behaviour of (1), while the physical parameters are uniquely estimated ~~(Section III-C)~~[M-mail].

III. Dual-Drive Gantry Physics Baseline

In this section, we give the equations of motion of the baseline (Section III-A), its LPV-LFR representation (Section III-B), and the identifiable combinations of its physical parameters (Section III-C).

A. Equations of motion

[**TODO:** state we leave out coulomb friction and that we will use for the experiment in the simulation in section V] The physics baseline of the gantry system used in this thesis is the lumped-parameter model of the dual-drive gantry of García-Herreros et al. [1]. Let L_b denote the cross-arm length between the two flexible joints. The generalised coordinates $q = [X\ \Theta\ Y]^\top$ are defined as $X = (X_1 + X_2)/2$ and $\Theta = (X_1 - X_2)/L_b$, the position and yaw angle of the cross-arm, while Y is unchanged. These coordinates avoid a closed kinematic-chain description and make the coupling associated with the nonuniform load distribution explicit [1]. [**TODO:** This needs to make clear why we choose for this realisation.] [**TODO:** How to connect, nonunfirom ... moving payload position] The coordinate and force transformations are

$$\begin{aligned} q = P^{-\top} q_{\text{act}}, \qquad u = P u_{\text{act}}, \\ P = \begin{bmatrix} 1 & 1 & 0 \\ L_b/2 & -L_b/2 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \end{aligned} \quad (3)$$

where u denotes the generalised force. Following García-Herreros et al. [1], the baseline assumes $\cos(\Theta) \approx 1$ and $\sin(\Theta) \approx 0$, valid for small yaw angles, and assumes that the Coriolis and centripetal force terms are negligible compared to the inertia, damping, and stiffness forces. García-Herreros et al. also model Coulomb friction at the actuators X_1, X_2 and Y [1, Sec. 2.2]. We leave this friction out of the baseline, so that the friction of the gantry (1) is part of the missing dynamics of Section II. [**TODO:** Check citation: Sec. 2.2] Under these assumptions, the baseline equations of motion are

$$M\big(Y(t)\big)\ddot{q}(t) + C\dot{q}(t) + Kq(t) = u(t), \quad (4)$$

with

$$M(Y) = \begin{bmatrix} m_1 + m_2 + m_b + m_h & \frac{L_b}{2}(m_1 - m_2) - m_h Y & 0 \\ \frac{L_b}{2}(m_1 - m_2) - m_h Y & J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2) + m_h(d^2 + Y^2) & -m_h d \\ 0 & -m_h d & m_h \end{bmatrix}, \quad (5)$$

$$C = \begin{bmatrix} c_{g1} + c_{g2} & \frac{L_b}{2}(c_{g1} - c_{g2}) & 0 \\ \frac{L_b}{2}(c_{g1} - c_{g2}) & c_{b1} + c_{b2} + \frac{L_b^2}{4}(c_{g1} + c_{g2}) & 0 \\ 0 & 0 & c_y \end{bmatrix}, \quad (6)$$

$$K = \begin{bmatrix} 0 & 0 & 0 \\ 0 & k_{b1} + k_{b2} & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

From here on the notation for the time dependency is dropped. [**TODO:** Proposed insertion: "García-Herreros et al. also model Coulomb friction at the three actuators [1, Sec. 2.2]. The baseline omits these terms, so that friction remains a physical effect the augmentation must capture (Section VI)." Reason documented in D-204. Second reason? A sign term is not twice differentiable, which [4, Cond. 8] assumes for well-posedness. because: ?] [**TODO:** Look at this section, the above state a lot, look fine] The masses, flexible-joint stiffness and damping coefficients, and the dimensions L_b and d are shown in Fig. 1. Here, J_b and J_h denote the yaw inertias of the cross-arm and the payload, while c_{g1}, c_{g2}, and c_y denote the actuator

damping coefficients. The masses m_1 and m_2 belong to the two X actuators, m_b to the cross-arm and m_h to the payload. The flexible joints have stiffness coefficients k_{b1}, k_{b2} and damping coefficients c_{b1}, c_{b2}, and d is the distance between the centres of mass of the payload and the cross-arm [1]. The nominal values are taken from [1, Table I] and listed in Appendix A-A. [**TODO:** Not sure if to add this, depends on real data or not; they were not identified on the present Telica machine.] To prevent the definition of q from being parameter-dependent, the cross-arm length L_b is fixed at the nominal value listed in Appendix A-A rather than treated as a free model parameter. [**TODO:** Js33 open ("Wording is a bit weird, in garcia they also do this but this implies they don't"): Garcia lists L_b in his Table I of identified parameters and does not state that he keeps it fixed, so "as in [3]" is not supported. Your wording decision.] ~~Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia matrix $M(Y)$. This position dependence is included by representing the baseline in linear parameter-varying (LPV) form, with the payload position Y as scheduling variable [11].~~[moved to III-B] ~~Freezing $M(Y)$ at $Y = 0$ increases the simulation error on X by up to a factor 3.7 (Fig. 12 in Appendix C).~~ [**TODO:** Determine and justify the operating range of Y; distinguish the mechanical stroke, simulated-data range, and range covered by the measured ILC records.]

B. Self-scheduled LPV-LFR representation

- Y is a state and the scheduling variable; $M(Y)^{-1}$ makes the model rational in Y (Drenth)
- $M(Y) = M_0 + YM_1 + Y^2M_2$; Y enters only through $Y\ddot{q}$ and $Y^2\ddot{q}$, realised by repeating Y (this work)
- Latent variables z, w and Delta-block $\Delta(Y) = YI_6$; not claimed minimal (this work)
- LTI part G in a continuous-time adaptation of Drenth Eq. (2); its first block row is the state equation
- Closing the algebraic loop gives back (4) exactly (this work)
- Well posed iff $M(Y)$ nonsingular, true for every Y; closed-form $M(Y)^{-1}$, no numerical solve (Drenth Def. 1; this work)

[from III-A] Moving the payload position Y changes the mass distribution of the gantry and therefore its inertia matrix $M(Y)$. This position dependence is included by representing the baseline in ~~linear parameter-varying (LPV)~~ LPV form, with the payload position Y as scheduling variable [11].[end] The payload position Y is both a state and the scheduling variable, making the baseline a self-scheduled LPV model.[**TODO:** source, or do i state it earlier..?] Because the scheduling variable is the state Y itself, this LPV form coincides with (4) for every motion of Y and needs no assumption on $\dot{Y}$. [**Note:** Make clear that the acceleration is obtainted from the formulas. Maarten also had a comment about this.] Solving (4) for $\ddot{q}$ requires $M(Y)^{-1}$, so the model is rationally dependent on Y. A linear fractional representation (LFR) is introduced to separate this rational scheduling dependence. We move the dependence out of the LTI part G and into a Delta-block $\Delta(Y)$, with the scheduling

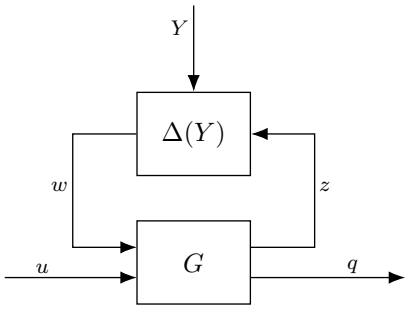


Fig. 3: Self-scheduled baseline interconnection. The Delta-block closes the latent variables of G through $w = \Delta(Y)z$, with $\Delta(Y) = YI_6$ and Y a component of the state.

being connected in feedback with G (Fig. 3) [4], [12]. To separate the scheduling variable Y, deconstruct the inertia matrix (5) into a polynomial in Y,

$$M(Y) = M_0 + YM_1 + Y^2M_2. \tag{7}$$

M_0, M_1 and M_2 are given in Appendix A-B. Substituting (7) into (4) gives

$$M_0\ddot{q} + YM_1\ddot{q} + Y^2M_2\ddot{q} = u - C\dot{q} - Kq. \tag{8}$$

Here Y only enters through $Y\ddot{q}$ and $Y^2\ddot{q}$, with $Y^2\ddot{q} = Y(Y\ddot{q})$. This lets us apply the same scheduling variable twice instead of introducing a second scheduling variable Y^2,

$$\ddot{q} \xrightarrow{Y} a_1 \xrightarrow{Y} a_2, \tag{9}$$

with $a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$. We treat a_1 and a_2 as independent signals, so (8) becomes

$$M_0\ddot{q} + M_1a_1 + M_2a_2 = u - C\dot{q} - Kq, \tag{10}$$

which contains no Y. This is allowed, because the Delta-block below imposes $a_1 = Y\ddot{q}$ and $a_2 = Ya_1$ again. The latent variables are $z = [\ddot{q}^\top \; a_1^\top]^\top$ and $w = [a_1^\top \; a_2^\top]^\top$, with the Delta-block mapping z to w,

$$w = \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} Y\ddot{q} \\ Ya_1 \end{bmatrix} = \underbrace{\begin{bmatrix} YI_3 & 0 \\ 0 & YI_3 \end{bmatrix}}_{\Delta(Y)=YI_6} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \Delta(Y)z. \tag{11}$$

[**TODO:** I have an SVD reduction add this in the appendix or not?] We write the LTI part G of the LFR in continuous time, adapting the discrete-time block structure of [12, Eq. (2)]: [**TODO:** Check this]

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = G \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}, \tag{12}$$

where $\tilde{x} = [q^\top \; \dot{q}^\top]^\top$ is the state and q the output. The first block row is the state equation, with $\ddot{q}$ obtained from (10) by multiplying with M_0^{-1}. [**Note:** This was not clear earlier for Maarten. At least where we compared the two systems for identifiability.] For the blocks of G, see Appendix A-B.
Conversely, closing the algebraic loop with (11) results in

$a_1 = Y\ddot{q}$ and $a_2 = Y^2\ddot{q}$, which turns (10) back into (4). The LFR is therefore an exact realisation of the baseline. The LFR is well posed if the algebraic loop has a unique solution for every Y [12, Def. 1]. For the gantry realisation we show that this condition holds when $M(Y)$ is nonsingular (Appendix A-B). For positive masses and inertias, $M(Y) \succ 0$ for every $Y \in \mathbb{R}$ (Appendix A-C). Therefore the LFR is well posed over the whole operating range of Y. [**TODO:** Connect the two sentences better?]

C. Physical parameters and identifiable combinations

- With q measured, known force scale and full actuation, the input-output behaviour fixes M_0, M_1, M_2, C, K and conversely (this work)
- Twelve nonzero entries, three equal to m_h, leave ten independent functions: exactly ten combinations are structurally identifiable (this work; definition Ovchinnikov)
- Written physically as θ_{base}, the gantry counterpart of base parameters (Gautier and Khalil); one-to-one with the entries for $m_h > 0$ (this work)
- Four lost directions; estimates reported in θ_{base}; result independent of the encoder; augmented model in Section IV; recovery from the records in Setup
- Positive θ_{base} does not give $M(Y) \succ 0$; that holds iff (14) (this work; proof Appendix A-C)
- Training keeps (14) at every iterate; the map and its justification are in Section IV-C

The physical parameters are estimated jointly with the augmentation ~~(Section IV)~~[M-mail], and interpretability requires that their estimates are unique (Section I). The fourteen parameters θ of [**TODO:** needs to have appendix in the name of the table] Table IV cannot all be identified from input-output data. The positions X_1, X_2, Y are measured, which gives q through (3), and the forces u are known in absolute scale. With unconstrained initial conditions and full actuation (P invertible), any acceleration $\ddot{q}$ can be reached at any state within the operating range. Since q is measured, the state $(q, \dot{q})$ is observable. Two parameter sets θ and θ' that produce the same motion for every input force satisfy

$$\big(M(Y) - M'(Y)\big)\ddot{q} + (C - C')\dot{q} + (K - K')q = 0$$

where $M(Y)$, C and K are evaluated at θ and $M'(Y)$, C' and K' at θ'. [**Note:** Do I need to state symbols like Quinten asked?] Therefore the identity must hold for all these states and accelerations, this implies that $K = K'$, $C = C'$ and $M(Y) = M'(Y)$. Since $M(Y)$ is quadratic in Y, this also implies that $M_0 = M_0'$, $M_1 = M_1'$ and $M_2 = M_2'$. If these matrices are equal, the input-output behaviour is also the same. Therefore, the part of θ that can be identified is determined by the parameter combinations in M_0, M_1, M_2, C and K. If we ignore the symmetric entries, (5) and (6) contain twelve nonzero entries. Three of these, $M_{0,33}$, $-M_{1,12}$ and $M_{2,22}$, are equal to m_h, where $M_{i,jk}$ denotes entry (j,k) of M_i. This leaves us with ten distinct functions of θ. The Jacobian of these ten functions with respect to $(m_h, d, m_1, m_b, J_b, c_{g1}, c_{g2}, c_{b1}, c_y, k_{b1})$ is block triangular and has a determinant of $m_h L_b^2/2 \neq 0$, meaning that the functions are independent. Since the matrices only depend on these ten functions, exactly ten independent combinations of θ can be identified [**TODO:** check source] [13, Def. 7]. We define these ten combinations as

$$\theta_{\text{base}} = [k_{b,\Sigma}, c_{g1}, c_{g2}, c_y, c_{b,\Sigma}, m_h, m_\Sigma, m_\Delta, J_{\text{eff}}, d]^\top, \quad (13)$$

with $k_{b,\Sigma} = k_{b1} + k_{b2}$, $c_{b,\Sigma} = c_{b1} + c_{b2}$, $m_\Sigma = m_1 + m_2 + m_b$, $m_\Delta = m_1 - m_2$ and $J_{\text{eff}} = J_b + J_h + \frac{L_b^2}{4}(m_1 + m_2)$. This follows the idea of base parameters used in robot dynamics [14].[**TODO:** check source] For $m_h > 0$, θ_{base} can be uniquely determined from the ten entries (Appendix A-D). We therefore estimate θ_{base} instead of θ~~, presented in the results~~.
Positive θ_{base} does not imply $M(Y) \succ 0$. For $m_h > 0$, $m_\Sigma > 0$ and $J_{\text{eff}} > 0$, $M(Y) \succ 0$ for every Y if and only if

$$m_\Sigma J_{\text{eff}} > \tfrac{L_b^2}{4}\, m_\Delta^2 \quad (14)$$

(Appendix A-C). To ensure that well-posedness and (14) hold during training, we use a logarithmic parametrisation for the nine positive combinations and a $\tanh$-bounded m_Δ ~~(Section IV-C)~~[M-mail]. [from IV-C, M-mail] The optimiser updates θ_{base} through training coordinates $\beta \in \mathbb{R}^{10}$, which are zero at the initial parameters θ^0_{base} and ~~keep every iterate admissible~~ ensure that (14) holds at every update. The nine positive parameter combinations$j$ of (13) are $\theta_{\text{base},j} = \theta^0_{\text{base},j} \exp(\beta_j)$, while m_Δ can be positive or negative and is bounded by (14):

$$m_\Delta = \gamma\, \frac{2}{L_b} \sqrt{m_\Sigma J_{\text{eff}}}\, \tanh\big(\eta_0 + s\,\beta_\Delta\big), \quad (15)$$

where β_Δ is the entry of β for m_Δ and $\gamma \in (0,1)$ provides a safety margin. We choose the offset η_0 and scale s so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta / \partial \beta_\Delta = m_{\Delta,0}$. This matches the logarithmic parameters, where $\partial \theta_{\text{base},j} / \partial \beta_j = \theta^0_{\text{base},j}$ at $\beta = 0$ (Appendix A-C). The initial value must satisfy $|m_{\Delta,0}| < \gamma(2/L_b)\sqrt{m_{\Sigma,0} J_{\text{eff},0}}$. The bound in (14) holds for every β, therefore no clamping or penalty is needed.[end]

IV. Dynamic Augmentation of the LPV-LFR Baseline

- The section extends the baseline with learned dynamics and identifies the result from closed-loop records; it names its five subsections (style profile)
- Contribution share: the realisation on the self-scheduled gantry baseline with joint estimation of θ_{base}, its identification in closed loop with the known controller, and the PS initialisation of the added states (MUST ESTABLISH)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

~~The dynamics of the system that are not captured by the baseline model defined in Section III-B will be identified from input-output data measured in closed-loop. The baseline model will be augmented to represent these missing dynamics.~~[moved to II, Jan7] ~~A correction to the states of the baseline model alone is not sufficient, because missing dynamics~~

~~need states the baseline does not have (Section IV-A), so additional states are introduced. We make use of a feedforward neural network to model both the changes for the baseline states and the additional states.~~[moved to II, Jan7] In this section, the baseline model of Section III is augmented to represent the missing dynamics introduced in Section II. We define the augmented model (Section IV-A), simulate it inside the known feedback loop (Section IV-B), estimate it jointly with θ_{base} (Section IV-C), and start the training from the baseline (Section IV-D) with a data-driven initialisation for the added states (Section IV-E).

A. Augmented model

- If the omitted effects are dynamic, the six baseline states cannot represent them; refitting θ_{base} or a static augmentation adds no states (EJC Sec. 2)
- Hoekstra's LFR augmentation structure with a fixed interconnection; the self-scheduled LPV-LFR is its baseline block; the learned block is a network, not Drenth's LPV-LFR augmentation (thesis Eq. (5.2)) (?: reason)
- Parallel because the orthogonal-by-construction method of Section IV is defined for the additive structure (Györök 2026); output map not augmented (Kessels Remark 5.3)
- The structure is a discrete-time state update (EJC Sec. 2, Table 1), so f_{base} is one RK4 step with the input held and $M(Y)$ per stage; the correction is added after the step
- Acyclic interconnection plus the baseline result of Section II: well posed for every Y (EJC Thm. 1)
- Normalisation: affine, statistics of the training records, velocities from differenced positions; later displays in normalised symbols
- One tanh MLP supplies f_{aug} and g_{aug} (EJC Sec. 4.3); it corrects every physical row, so a persistent correction on X or Y accumulates in position (?: justification); $n_{\bar{x}} = 0$ is the static parallel augmentation
- The baseline/network split is not unique (2026 Sec. 5.2); interpretable means the estimated θ_{base} keeps its physical meaning (Györök Sec. 3); Section IV makes the estimate unique

To represent additional dynamics such as a flexible mode, refitting θ_{base} or correcting the baseline states with a static augmentation , where the learned correction is only added to the physical states, is not sufficient. Both these methods only retain the original states of the baseline~~, while missing dynamics require states the baseline does not include~~ [4, Sec. 2]. A dynamic augmentation therefore adds states $\bar{x}$ to model these missing dynamics. The augmented model updates these states using the full state and input [4, Sec. 2, Table 1].

We use the dynamic parallel structure of Hoekstra et al. [**TODO:** [4, Table 1]] (Fig. 4), here a neural network adds it correction on the same state as the baseline model in parallel. besides the baseline model on the same state. The learning components are a neural network consisting of f_{aug} and g_{aug}. The model is defined in discrete time with the augmentation added to the state update of the baseline [4, Sec. 2, Table 1]. This gives us

$$\begin{aligned}\tilde{x}_{k+1} &= f_{\text{base}}(\theta_{\text{base}}, \tilde{x}_k, u_{\text{act},k}) + f_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \bar{x}_{k+1} &= g_{\text{aug}}(\theta_{\text{aug}}, \hat{x}_k, u_{\text{act},k}),\\ \hat{y}_k &= P^\top q_k,\end{aligned} \tag{16}$$

where $\tilde{x}_k = [q_k^\top \ \dot{q}_k^\top]^\top \in \mathbb{R}^6$ is the physical state of (12), $\bar{x}_k \in \mathbb{R}^{n_{\bar{x}}}$ the added state, the augmented state $\hat{x}_k = [\tilde{x}_k^\top \ \bar{x}_k^\top]^\top$, f_{base} the baseline state update, $u_{\text{act},k} \in \mathbb{R}^3$ the actuator forces, $\hat{y}_k \in \mathbb{R}^3$ the predicted actuator positions, and y_k the measured positions of (1), $\theta_{\text{base}} \in \mathbb{R}^{10}$ the combinations (13), and [**TODO:** using theta might be confusing with baseline] $\theta_{\text{aug}} \in \mathbb{R}^{n_{\text{aug}}}$ the network parameters. The added states are updated by g_{aug} without any predetermined physical meaning, in contrast to the baseline states $\tilde{x}$ for which the positions are measured.

The dynamic-parallel augmentation structure is chosen, because the orthogonal-by-construction ~~parametrisation that Section V builds upon~~ augmentation of Section I[M-mail] is defined for this additive structure [7].

The output map is not augmented ($h_{\text{aug}} = 0$ in Fig. 4), because (3) gives the exact relation between the states and the measured outputs , as in Kessels [9, Remark 5.3]. This is also done to ensure that q still represents the physical positions. With a learned correction to the output map the states could stop representing the actual positions. [**TODO:** not an LPV-LFR with a learned scheduling map as in the augmentation of Drenth [15, Eq. (5.2)].] [**TODO:** Missing: reason for a network as learning block rather than Drenth's LPV-LFR augmentation. The research plan (Aspect 2) extends the structure "to the LPV setting following" Drenth and leaves "the specific parameterization to be established". Candidates: (a) Hoekstra's framework with its zero start and well-posedness result is kept as is; (b) the research question asks only for a structure that captures the omitted dynamics. Close D-017 with a decision entry.]

The baseline is evaluated over one sample period T_s. Writing (4) in state-space form givesthe state derivative F

$$\dot{\tilde{x}} = F(\tilde{x}, u) = \begin{bmatrix} \dot{q} \\ M(Y)^{-1}\big(u - C\dot{q} - Kq\big) \end{bmatrix}, \tag{17}$$

where $u = Pu_{\text{act}}$ is the generalised force (3), M, C and K depend on θ_{base} through (13), and $M(Y)^{-1}$ is the closed loop of the LFR of Section III-B. f_{base} is calculated using one fourth-order Runge-Kutta (RK4) step of F over T_s, with the input held constant (zero-order hold) during the step. For each of the four RK4 sub-steps $M(Y)$ is evaluated using the value of Y from its intermediate state.

[**TODO:** The augmented model admits a unique solution for every state, input and Y. The interconnection has no algebraic loops, making it acyclic, and both blocks are twice continuously differentiable, which together give well-posedness [4, Cond. 6, Cond. 8, Thm. 9]. The baseline LFR is also well posed (Section III-B).]

The inputs to the neural network are normalised to zero mean and unit variance, which has been shown to improve the

estimation of models containing neural networks [4, Sec. 5.3]. The mean and standard deviations are computed from the training data. For the baseline states the measured positions are converted to their generalised positions, for which the velocities can be estimated from their finite difference by dividing with T_s. This ensures that the positions and outputs are scaled with the same measurements. This also makes sure that all models independent of its initial parameters use the same scale,

$$\tilde{x}^{\mathrm{n}} = T_x(\tilde{x} - \mu_x), \quad u^{\mathrm{n}} = T_u(u_{\mathrm{act}} - \mu_u), \quad y^{\mathrm{n}} = T_y(y - \mu_y), \tag{18}$$

where $\mu_x \in \mathbb{R}^6$ and $\mu_u, \mu_y \in \mathbb{R}^3$ are the channel means, and $T_i = \mathrm{diag}(\sigma_i)^{-1}$ for $i \in x, u, y$, with σ_i the corresponding standard deviations. The additional states $\bar{x}$ are not normalised separately, as their initial scale is provided by the neural-network initialisation and the normalised input data, [**TODO:** The additional states $\bar{x}$ are not normalised separately. Their scale is learned by the encoder and the added-state transition network, operating on normalised physical signals.] so $\hat{x}^{\mathrm{n}} = [\tilde{x}^{\mathrm{n}\top}\ \bar{x}^{\top}]^{\top}$.

After normalisation the model equations become

$$\begin{aligned} f^{\mathrm{n}}_{\mathrm{base}}(\theta_{\mathrm{base}}, \tilde{x}^{\mathrm{n}}, u^{\mathrm{n}}) &= T_x\big(f_{\mathrm{base}}(\theta_{\mathrm{base}}, \tilde{x}, u_{\mathrm{act}}) - \mu_x\big), \\ f^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}, u^{\mathrm{n}}) &= T_x f_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}, u_{\mathrm{act}}), \\ g^{\mathrm{n}}_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}, u^{\mathrm{n}}) &= g_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}, u_{\mathrm{act}}), \\ \hat{y}^{\mathrm{n}}_k &= C^{\mathrm{n}} \tilde{x}^{\mathrm{n}}_k, \qquad C^{\mathrm{n}} = T_y P^{\top} [I_3\ \ 0]\, T_x^{-1}. \end{aligned} \tag{19}$$

where $\tilde{x}$ and u_{act} follow from $\tilde{x}^{\mathrm{n}}$ and u^{n} by inverting (18), so RK4 uses the states and inputs in SI units. Because f_{aug} is added to f_{base}, whose mean μ_x is already subtracted in $f^{\mathrm{n}}_{\mathrm{base}}$, the correction is only scaled by T_x. [**TODO:** The correction changes the predicted state, so its normalised form is $f^{\mathrm{n}}_{\mathrm{aug}} = T_x f_{\mathrm{aug}}$, without subtracting μ_x. In the implementation, the network directly outputs $f^{\mathrm{n}}_{\mathrm{aug}}$.] The output map remains linearwithout offset because the mean of the positionsin μ_x is calculated from the output mean as $P^{-\top}\mu_y$.

A multilayer perceptron ϕ_{aug} with tanh activation functions for the hidden layers and a linear output layer is used for both $f^{\mathrm{n}}_{\mathrm{aug}}$ and $g^{\mathrm{n}}_{\mathrm{aug}}$:

$$\left[f^{\mathrm{n}\top}_{\mathrm{aug}}\ \ g^{\mathrm{n}\top}_{\mathrm{aug}}\right]^{\top} = \phi_{\mathrm{aug}}(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_k, u^{\mathrm{n}}_k).$$

The inputs to the neural network are the six baseline states and $n_{\bar{x}}$ added states and the $n_u = 3$ actuator forces. The first six outputs $f^{\mathrm{n}}_{\mathrm{aug}}$ update the six physical states and the remaining $n_{\bar{x}}$ outputs $g^{\mathrm{n}}_{\mathrm{aug}}$ equal the additional states $\bar{x}_{k+1}$ (Fig. 4). These corrections act on all six physical states, the positions included, as in the dynamic-parallel structure [4, Table 1]. The positions are also corrected because within one sample period an unmodelled force would change both the velocity and position, correcting only for the velocities would not be able to capture this. Both f_{aug} and g_{aug} are given the payload position Y through $\tilde{x}_k$, enabling the neural network to learn position-dependent effects. ~~The gantry baseline model has two axes, X and Y, without stiffness to ground in (6), which are also affected by the learned correction. The linearised RK4 step has a pole at $z = 1$ on each of these axes, making it marginally stable: stable, but not asymptotically stable. Therefore these axes act as integrators: a constant error in the correction keeps accumulating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. The feedback controller is able to correct this drift, which is why we have chosen to simulate the model in closed-loop (Section IV-B).~~ moved to IV-B, M-p6

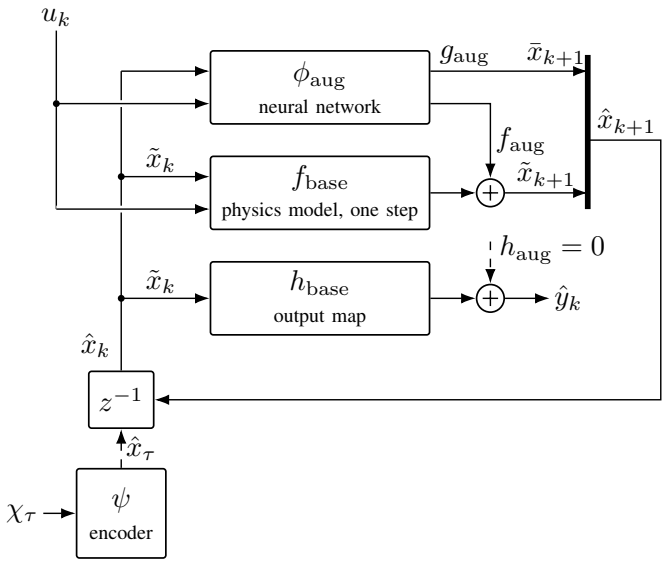


Fig. 4: Augmented model. The physics step f_{base} and the network ϕ_{aug} act in parallel on $\hat{x}_k$. ϕ_{aug} gives f_{aug} and g_{aug}, and the output map is not learned. The encoder ψ, introduced in Section IV-C, sets $\hat{x}_\tau$ at the start $k = \tau$ of each window. f_{base} and h_{base} use the physical part $\tilde{x}_k$ of $\hat{x}_k$.[**TODO:** No setpoint generator currently for the inputs; layout cramped at one column width.]

B. *Closed-loop simulation*

- The model is not driven by the recorded input (2026 Eq. (22)), for two reasons: noise-induced bias under feedback (Kessels Remark 5.6, SUBNET; indirect approach, Forssell and Ljung) and the free axes of Section IV-A, which violate the stability condition behind SUBNET consistency (Beintema Cond. 1); motivation, not a proof
- As in Kessels (Eqs. (5.12), (5.13)), each window is simulated in closed loop with the known controller, which forms its error against the model output; system and model loops share r and u^{ff}
- The controller is linear and time invariant, so subtracting the system's loop cancels r and u^{ff}; the residual form needs only the recorded u_{act} and y, and is exact when u_{act} was produced by this controller from y (this work)
- $x^c_\tau = 0$ starts the model's controller in the system's controller state, as the encoder does for the plant state; no controller state is estimated, unlike Kessels Remark 5.4 (this work)

- No plant feedthrough fixes the step order, so D_{fb} closes no algebraic loop (as Kessels Remark 5.1); one normalised step displayed
- The output error re-enters the model input, so part of it is corrected within the window; error before τ is not carried in, the free-run validation carries it

The data is measured in closed-loop, and the model is simulated in closed-loop. Instead of using the recorded input as in [4, Eq. (22)], we include the feedback controller in the simulation ~~for two reasons~~[Jan49]. ~~First, as established in Section IV-A: the axes X and Y without stiffness make the open-loop gantry marginally stable.~~[M-p6 from IV-A, M-p6] The gantry baseline model has two axes, X and Y, without stiffness to ground in (6), which are also affected by the learned correction. The linearised RK4 step has a pole at $z = 1$ on each of these axes, making it marginally stable: stable, but not asymptotically stable. Therefore these axes act as integrators: a constant error in the correction keeps accumulating in the position. The alternative to only correct the velocities would not prevent the integrator issue, because an error in the velocity also accumulates in the position. The feedback controller is able to correct this drift~~, which is why we have chosen to simulate the model in closed-loop (Section IV-B).~~ .[M-p6 end] ~~SUBNET, the encoder-based identification method of Beintema et al. [16], assumes that the system that generates the data is incrementally exponentially output stable [16]. The identification method we have adapted from [4] relies on this SUBNET assumption [4]. The open-loop gantry system does not satisfy this assumption: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset.~~[moved to IV-C, M-p6]

~~Second, the feedback controller causes a correlation between the recorded input and the measurement noise. Kessels was not able to identify an open-loop SUBNET model using closed-loop data from a wire bonder. Kessels cites earlier work suggesting that this could be explained by noise-induced bias due to this correlation and the integral term of the controller [9].~~[Jan49] ~~This bias is avoided in the indirect approach for linear systems with a known controller, which fits the closed-loop model using the reference. Because the reference is uncorrelated with the noise, the model can be identified from the output error [17].~~[M-p6] ~~These reasons motivate~~ This motivates[Jan49] the choice for simulating in closed-loop. ~~They do not prove that our estimate is consistent.~~[Jan49] [**TODO:** Missing: evidence on the thesis data that open-loop rollouts accumulate position error on X and Y. Candidate: the open-loop versus closed-loop drift on one record that `THESIS-RESULTS.md` Sec. 5 plans, reported in Section VII.]

~~Every training window is~~ The model is[M-p7] simulated in closed-loop with the assumed known controller, as in Kessels [9, Eqs. (5.12), (5.13)]. The feedback controller is linear and time invariant, the model output is used to calculate the position error in actuator coordinates[**TODO:** qact?]. The controller calculates the actuator forces from this error. The system and model use the same reference r and feedforward u^{ff}, but use their own output for feedback, with controller states x^{fb} and $\hat{x}^{\mathrm{fb}}$ (Fig. 5), with the system loop of (2)[M-p3]: ~~[system rows of this equation moved to (2)]~~[moved to II, M-p3]

$$\begin{aligned} \hat{x}^{\mathrm{fb}}_{k+1} &= A_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + B_{\mathrm{fb}}(r_k - \hat{y}_k), \\ \hat{u}_{\mathrm{act},k} &= C_{\mathrm{fb}}\hat{x}^{\mathrm{fb}}_k + D_{\mathrm{fb}}(r_k - \hat{y}_k) + u^{\mathrm{ff}}_k, \end{aligned} \tag{20}$$

~~where $(A_{\mathrm{fb}}, B_{\mathrm{fb}}, C_{\mathrm{fb}}, D_{\mathrm{fb}})$ are the state-space matrices of the controller discretised with sampling time T_s. Both r_k and u^{ff}_k are in actuator coordinates.~~[moved to II, M-p3] The model uses the calculated actuator force $\hat{u}_{\mathrm{act},k}$ instead of the recorded $u_{\mathrm{act},k}$ in (16).

Since we use a linear controller we are able to subtract the system and model controller equations to cancel r and u^{ff}. This leaves us with the controller state difference $x^c = \hat{x}^{\mathrm{fb}} - x^{\mathrm{fb}}$, driven by $y - \hat{y}$. The model is still simulated with respect to the reference r, but r and u^{ff} are not used in simulation. If the controllers between the system and simulation differ, the equations are only approximate and noise-induced bias cannot be ruled out.

At the start of ~~each training window~~ a simulation at sample τ[M-p7] we set $x^c_\tau = 0$, which sets the model's controller state equal to that of the system at τ. ~~Therefore the encoder only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [9].~~[moved to IV-C, M-p7] We then write the closed-loop model in (20) as

$$\begin{aligned} x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}(y_k - \hat{y}_k), \qquad x^c_\tau = 0, \\ \hat{u}_{\mathrm{act},k} &= u_{\mathrm{act},k} + C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}(y_k - \hat{y}_k), \end{aligned} \tag{21}$$

which only needs the recorded u_{act} and y. [**TODO:** Should be discussed in the setup for controller discretisation] [**TODO:** Check for Setup: the training loop uses the controller re-discretised at the model rate, while the records were generated at a higher rate. Run the re-discretised controller on the recorded $r - y$ and compare its feedback force with the recorded one. A rollout with $\hat{y} = y$ gives $\hat{u} = u$ by construction and checks nothing. Related: `FINAL-RUN-FIXES.md` item 3, the D-232 leak check on the lowpass data, passed (0.22 to 0.50 nm).]

The model output $\hat{y}_k$ depends only on the state $\tilde{x}_k$, not on the input $\hat{u}_k$. For each sample we first calculate the output and then the error and model input before updating the states. This avoids an algebraic loop through D_{fb}, for the same reason as Kessels [9, Remark 5.1]. For a ~~window~~ simulation[M-p7] starting at τ, the model equations in the normalised coordinates of (18) are

$$\begin{aligned} \hat{y}^{\mathrm{n}}_{k|\tau} &= C^{\mathrm{n}}\tilde{x}^{\mathrm{n}}_{k|\tau}, \qquad \varepsilon_{k|\tau} = T_y^{-1}\big(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\big), \\ \hat{u}^{\mathrm{n}}_{k|\tau} &= u^{\mathrm{n}}_k + T_u\big(C_{\mathrm{fb}}x^c_k + D_{\mathrm{fb}}\varepsilon_{k|\tau}\big), \\ x^c_{k+1} &= A_{\mathrm{fb}}x^c_k + B_{\mathrm{fb}}\varepsilon_{k|\tau}, \qquad x^c_\tau = 0, \end{aligned} \tag{22}$$

and the next model state $\hat{x}^{\mathrm{n}}_{k+1|\tau}$ is calculated using (19) with input $\hat{u}^{\mathrm{n}}_{k|\tau}$. ~~Each window starts from the encoder state in (24).~~[M-p8] ~~The controller uses the output error to correct the model input within the training window, as in the closed loop system. Each window starts from a new encoder estimate and~~

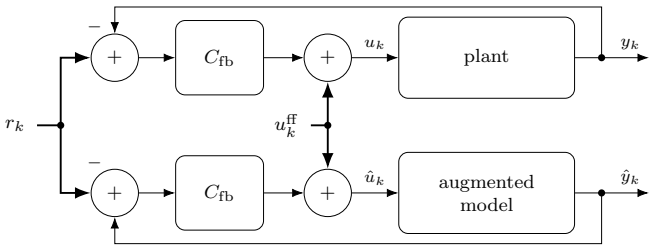


Fig. 5: Recorded system loop (top) and augmented model loop (bottom), both using the same feedback controller, reference r_k and feedforward u_k^{ff}. The model output is calculated before the input updates the model state.

~~$x_\tau^c = 0$, so the window is not affected by model errors before τ. The validation of Section IV-C is simulated in closed-loop and only initialised at the start.~~ moved to IV-C, M-p8

C. Identification

- θ_{base} is estimated jointly with network and encoder, as in 2026 Eq. (22); Kessels trains only the network weights (Eq. (5.12))
- Training coordinates β, zero at θ^0_{base}: nine combinations logarithmic, m_Δ through (15); every β admissible, so no clamp or penalty (this work, D-229; adapted from Drenth Sec. 4.2)
- Criterion (23): Eq. (22) adapted by the loop of Section IV-B and the encoder of Section IV-D; every window sample scored; controller fixed; a closed-loop fit can mask plant error (Section VI)
- The baseline/network split is not unique (2026 Sec. 5.2), so θ_{base} is not interpretable in the sense of Section I; Section IV addresses this (unique to first order only?)
- Adam, then an unbounded L-BFGS polish (adapted from Drenth Sec. 4.1); checkpoint with the lowest closed-loop free-run error over whole validation records; polish kept only if it lowers that error
- Initialisation (28): $\tilde{x}$ and $\hat{y}$ reproduce the baseline, $\bar{x}$ evolves through a live g_{aug} (2026 Eq. (29)); adapted from Sec. 5.4.3 (random added-state rows, no linear bypass); the model is named in Section VI, not here

This subsection states how the augmented model is estimated from the records: which parameters are trained, the criterion, the encoder that gives each simulation window its initial state, and the optimiser. from II, Jan23 The model is trained using short windows of the closed-loop data, where each window is simulated with the known controller. The initial states are estimated by ~~an encoder~~ an encoder network[Q37], including the additional states. During training, the physical parameters θ_{base} are also estimated, using the combinations defined in Section III-C.end This is the SUBNET approach of Beintema et al. [16]: the simulation error over short windows is minimised, and an encoder, trained jointly with the model, estimates the initial state of each window from past inputs and outputs.

~~The network parameters θ_{aug}, the encoder parameters θ_{enc} and θ_{base} are jointly estimated, as in [4], while the encoder paper keeps θ_{base} fixed [18].~~ moved to IV-C, below, M-p8 ~~The optimiser updates θ_{base} through training coordinates $\beta \in \mathbb{R}^{10}$, which are zero at the initial parameters θ^0_{base} and keep every iterate admissible (Section III-C). The nine positive parameter combinations j of (13) are $\theta_{\mathrm{base},j} = \theta^0_{\mathrm{base},j}\exp(\beta_j)$, while m_Δ can be positive or negative and is bounded by (14): [equation (15) moved to Section III-C] where β_Δ is the entry of β for m_Δ and $\gamma \in (0,1)$ provides a safety margin. We choose the offset η_0 and scale s so that $\beta = 0$ gives $m_\Delta = m_{\Delta,0}$ and $\partial m_\Delta/\partial\beta_\Delta = m_{\Delta,0}$. This matches the logarithmic parameters, where $\partial\theta_{\mathrm{base},j}/\partial\beta_j = \theta^0_{\mathrm{base},j}$ at $\beta = 0$ (Appendix A-C). The initial value must satisfy $|m_{\Delta,0}| < \gamma(2/L_b)\sqrt{m_{\Sigma,0}J_{\mathrm{eff},0}}$. The bound in (14) holds for every β, therefore no clamping or penalty is needed.~~ moved to III-C, M-mail ~~The parameters $\vartheta = (\beta, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the controller fixed. We adapt this from [4], adapted for the closed loop of Section IV-B and the encoder described below: [criterion (23) moved below] for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$ and has length n_f, $n_y = 3$, and $\theta_{\mathrm{base}}(\beta)$ gives the baseline parameters. $W_r = \mathrm{diag}(\sigma_{y,j}/\sigma_{e,r,j})$ weights axis j of training record r, so that its residual in metres is divided by the standard deviation $\sigma_{e,r,j}$ of its servo error (44).~~ moved to IV-C, below, M-p8

Each simulation window starts at a sample τ from an initial state $\hat{x}_\tau$, and k denotes the sample within the window, but we measure only the positions of the physical state $\tilde{x}$. The additional states $\bar{x}$ are latent, yet the simulation of each window needs their value at τ. Estimating $\hat{x}_\tau$ as a separate parameter for each window would increase the number of parameters as the data grows [16, Sec. 3.1]. Following Beintema et al., we use an encoder network that estimates $\hat{x}_\tau$ from past inputs and outputs [16, Sec. 3.2]. Simulating the windows of n_f samples using the encoded states keeps the computation per gradient step independent of the record length for fixed batch size and window lengths [16, Sec. 3.1, Eq. (9)]. Using short simulation windows increases the loss function smoothness [16, Sec. 3.1]. from IV-C start, M-p8 The network parameters θ_{aug}, the encoder parameters θ_{enc} and θ_{base} are jointly estimated, as in [4, Eq. (22)], while the encoder paper keeps θ_{base} fixed [18, footnote 1].end from IV-C start, M-p8 The parameters $\vartheta = (\beta, \theta_{\mathrm{aug}}, \theta_{\mathrm{enc}})$ are trained to minimise the normalised output error of the closed-loop simulation, with the controller fixed. We adapt this from [4, Eq. (22)], adapted for the closed loop of Section IV-B and the encoder described below:

$$\min_{\vartheta} V(\vartheta) = \frac{1}{|\mathcal{W}|\, n_f\, n_y} \sum_{\tau\in\mathcal{W}} \sum_{k=\tau}^{\tau+n_f-1} \left\| W_r\left(y^{\mathrm{n}}_k - \hat{y}^{\mathrm{n}}_{k|\tau}\right)\right\|_2^2 \quad \text{s.t.}$$
$$\tilde{x}^{\mathrm{n}}_{k+1|\tau} = f^{\mathrm{n}}_{\mathrm{base}}\left(\theta_{\mathrm{base}}(\beta), \tilde{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\right) + f^{\mathrm{n}}_{\mathrm{aug}}\left(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\right),$$
$$\bar{x}_{k+1|\tau} = g^{\mathrm{n}}_{\mathrm{aug}}\left(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k|\tau}, \hat{u}^{\mathrm{n}}_{k|\tau}\right),$$
$$\hat{x}^{\mathrm{n}}_{\tau|\tau} = \psi(\theta_{\mathrm{enc}}, \chi_\tau), \quad \hat{y}^{\mathrm{n}}_{k|\tau},\ \hat{u}^{\mathrm{n}}_{k|\tau} \text{ from (22)}, \tag{23}$$

for $\tau \le k < \tau + n_f$, where each training window starts at a sample $\tau \in \mathcal{W}$ and has length n_f, $n_y = 3$, and $\theta_{\mathrm{base}}(\beta)$ gives the baseline parameters. $W_r = \mathrm{diag}(\sigma_{y,j}/\sigma_{e,r,j})$ weights

axis j of training record r, so that its residual in metres is divided by the standard deviation $\sigma_{e,r,j}$ of its servo error ~~(44)~~ $y_j - r_j$[M-mail]. [**TODO:** might have to move to discussion if i dont add it. Since the controller corrects part of the output error, it can hide errors in the plant model. We therefore also test the model with a different controller in Section VII.][end]

~~The encoder uses~~ In (23), the encoder ψ uses[M-p8] the samples up to $\tau - 1$, as in [4, Eq. (22e)]:

$$\begin{aligned} \hat{x}^{\mathrm{n}}_{\tau|\tau} &= \psi(\theta_{\mathrm{enc}}, \chi_\tau), \\ \chi_\tau &= \big[\, (y^{\mathrm{n}}_{\tau-n-1})^\top \ \cdots \ (y^{\mathrm{n}}_{\tau-1})^\top \\ &\qquad (u^{\mathrm{n}}_{\tau-n-1})^\top \ \cdots \ (u^{\mathrm{n}}_{\tau-1})^\top \big]^\top, \end{aligned} \tag{24}$$

where $\hat{x}^{\mathrm{n}}_{\tau|\tau}$ is the normalised state $\hat{x}_\tau$ of Fig. 4 at the start of the window, $n+1$ is the number of samples in the window, $\chi_\tau \in \mathbb{R}^{6(n+1)}$ stacks the normalised recorded outputs and inputs as in (18), and $\theta_{\mathrm{enc}} \in \mathbb{R}^{n_{\mathrm{enc}}}$ is the parameter vector of the encoder. The encoder estimates all $6 + n_{\bar{x}}$ initial states $\hat{x}_\tau$, which includes the measured positions. We estimate the positions instead of using the measured y_τ to keep the positions consistent with the estimated velocities.

The encoder uses a linear map with a nonlinear correction [18, Eq. (8)]:

$$\psi(\theta_{\mathrm{enc}}, \chi_\tau) = \begin{bmatrix} W^b \\ W^a \end{bmatrix} \chi_\tau + \tilde{\psi}(\chi_\tau), \tag{25}$$

where $W^b \in \mathbb{R}^{6\times 6(n+1)}$ and $W^a \in \mathbb{R}^{n_{\bar{x}}\times 6(n+1)}$ are the linear maps for the six physical states and the $n_{\bar{x}}$ additional states, and $\tilde{\psi}$ is a neural network. The encoder parameters θ_{enc} are W^b, W^a and the parameters of $\tilde{\psi}$. The first six outputs ψ_b estimate the physical states and the remaining $n_{\bar{x}}$ outputs ψ_a estimate the additional states. We use θ^a_{enc} for the parameters that affect only ψ_a: W^a and the additional-state rows of the output layer of $\tilde{\psi}$. [**TODO:** VERIFY bib: `hoekstra2026encoder` = arXiv:2602.13108v1 (13 Feb 2026), Hoekstra, Györök, Tóth, Schoukens; matches PDF p. 1. Check for a published (IFAC) version before submission.]

[from IV-B, M-p7] SUBNET~~, the encoder-based identification method of Beintema et al. [16],~~[M-p7] assumes that the system that generates the data is incrementally exponentially output stable [16, Cond. 1]. The identification method we have adapted from [4] relies on this SUBNET assumption [4, Sec. 5.5]. The open-loop gantry system does not satisfy this assumption: a mismatch in the initial velocity will create a permanent position offset. In closed-loop the feedback controller is able to correct this offset, which is why each window is simulated in the closed loop of Section IV-B[M-p7].[end]

[from IV-B, M-p7] ~~Therefore the~~ With $x^c_\tau = 0$, the[M-p7] encoder only has to initialise the states of the model. Kessels instead needs the system's controller state at the start of each window, either measured directly or calculated by simulating the controller from the start of the record [9, Remark 5.4].[end] [from IV-B, M-p8] The controller uses the output error to correct the model input within the training window, as in the closed loop system. Each window starts from a new encoder estimate and $x^c_\tau = 0$, so the window is not affected by model errors before τ. The validation ~~of Section IV-C~~[M-mail] is simulated in closed-loop and only initialised at the start.[end]

The baseline parameters and the neural network can both correct the same model errors, making the optimal parameter values not unique [4, Sec. 5.2]. [**TODO:** Missing: Section V makes the network orthogonal to the linearised baseline sensitivity at fixed tuples, so the split is unique only to first order around β_{lin}, not a unique minimiser of (23). Candidate: once Section V states that local result, write "so that the split is unique to first order" with a pointer to it (basis: `04_obc.tex` (38); math check 2026-10-13, F1).]

For training we minimise V with Adam on minibatches of windows, and the gradient is calculated over the full window. We then polish with L-BFGS on a fixed set of windows. This is adapted from the Adam and L-BFGS-B sequence of Drenth et al. [12, Sec. 4.1]. Because every β satisfies (14), unconstrained L-BFGS works without a bound. During the optimisation with Adam the parameters with the lowest validation error are kept. The error is the mean ~~$\mathrm{NRMS}_{e,j}$ (44)~~ RMS simulation error divided by the RMS servo error, $\mathrm{NRMS}_{e,j}$,[M-mail] over the complete validation records and axes, simulated in closed loop with the initial state estimated once by the encoder and $x^c = 0$. We only keep the polish if it lowers the validation error.

D. Initialisation

- Only the positions are measured, so an encoder estimates all $6 + n_{\bar{x}}$ states at each window start from the input-output window (Beintema Sec. 3.1); the window ends at $\tau - 1$ (2026 Eq. (22e), D-250)
- The encoder is a linear map plus a network correction, as encoder paper Eq. (8), with no separate bias (the output bias of $\tilde{\psi}$ carries any constant)
- The physical rows start data-based: the baseline is self-scheduled by the state Y, hence nonlinear; a reconstruction from a linearisation is accurate only near its linearisation point (encoder paper Sec. 3.1), and the paper suggests the data-based route for nonlinear baselines (Secs. 3.2, 4.4)
- The physical rows are fitted to a baseline simulation as in 2026 Eq. (30); the simulation runs in closed loop because the open-loop baseline drifts on X and Y (this work, D-251)
- The fitted physical rows only set the start and are trained jointly with θ_{base} (2026 Eq. (22)); least-squares start (D-271)

A fully random initialisation can be unstable and give poor optimisation results. To resolve this, training starts from a model that behaves as the baseline [4, Sec. 5.4].

The physical rows are initialised from the baseline model. Because solving (4) for $\ddot{q}$ requires $M(Y)^{-1}$ and Y is a state (Section III-B), the baseline is nonlinear. To create a model-based state reconstruction map for the nonlinear baseline, a linearisation point has to be chosen, making the map only accurate around that point [18, Sec. 3.1]. Hoekstra et al. therefore suggest fitting the encoder to states simulated with the baseline [18, Sec. 3.2, Eq. (35)], as their augmentation method does [4, Eq. (30)]. We apply this approach and simulate the baseline model once before training to obtain the physical

state at every sample τ. The recorded actuator forces u_{act} and positions y from the training records with the known controller in (22) are used to simulate the physical states in closed-loop:

$$\tilde{x}^{\mathrm{n},0}_{k+1} = f^{\mathrm{n}}_{\mathrm{base}}\big(\theta^{0}_{\mathrm{base}}, \tilde{x}^{\mathrm{n},0}_{k}, \hat{u}^{\mathrm{n},0}_{k}\big), \qquad \hat{y}^{\mathrm{n},0}_{k} = C^{\mathrm{n}}\tilde{x}^{\mathrm{n},0}_{k}, \quad (26)$$

where $\tilde{x}^{\mathrm{n},0}_{k}$ is the simulated baseline state, $\theta^{0}_{\mathrm{base}}$ are the initial parameters, and $\hat{u}^{\mathrm{n},0}_{k}$ is the closed-loop input from (22). We use $\hat{y}^{\mathrm{n},0}_{k}$ as the model output and set the controller state x^{c} to zero at the start of each record (see Section IV-B). For our closed-loop simulation, each record starts from the measured positions in generalised coordinates, $q = P^{-\top}y$, with zero velocities, zero added states and $x^{c} = 0$. [**TODO:** Check what Hoekstra et al. use as the initial state of the baseline simulation that generates the encoder-fitting data (2026 Sec. 5.4.2, encoder paper Sec. 3.2, and their public code), and whether our records start at rest, which the zero velocities assume.] The physical rows of the encoder are then fitted to the simulated states by least squares [18, Eq. (33)]:

$$\big(W^{b,0}, b^{b,0}\big) \in \arg\min_{W^{b},\, b} \frac{1}{|\mathcal{D}|} \sum_{\tau\in\mathcal{D}} \big\| W^{b}\chi^{0}_{\tau} + b - \tilde{x}^{\mathrm{n},0}_{\tau} \big\|_{2}^{2}, \quad (27)$$

where $b \in \mathbb{R}^{6}$ is an intercept. Hoekstra et al. use this linear fit for LTI baselines and fit a neural network for nonlinear baselines [18, Eqs. (33), (35)]. We use the linear fit as a linear restriction of their data-based initialisation, and learn the nonlinear correction $\tilde{\psi}$ during training. The affine fit is computed by a single numerical-rank-truncated SVD solve in centred and standardised regressors, without gradient-based pre-training. The intercept $b^{b,0}$ is placed in the output bias of the physical rows of $\tilde{\psi}$, whose output weights start at zero, so that $\psi_{b} = W^{b,0}\chi_{\tau} + b^{b,0}$ at the start. The hidden layers of $\tilde{\psi}$ are drawn from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer. We write $\theta^{b,0}_{\mathrm{enc}}$ for this start of $\theta^{b}_{\mathrm{enc}}$, which consists of W^{b}, the hidden layers of $\tilde{\psi}$ and its physical output rows, and use it to start minimising (23). χ^{0}_{τ} is formed as we do in (24) using $\hat{y}^{\mathrm{n},0}$ and $\hat{u}^{\mathrm{n},0}$. $\mathcal{D}$ is the set of window ends τ at the training stride of 10 samples. We use the closed-loop input from (26), instead of the recorded input as in [4, Eq. (30)]. The fitting loss in (27) and $\theta^{b,0}_{\mathrm{enc}}$ are only used for initialisation. Subsequent training uses (23).

We start training with the augmented model giving the same output as the baseline model from the same initial state, as posed by [4, Eq. (29)]. [**Note:** I use the encoder for my augmented model, so i state: model from the same initial state. But i could leave it out.] Therefore we zero the physical rows of ϕ_{aug} its output layer, and the added-state rows are kept random:

$$\begin{aligned}
&\text{baseline:} && \beta = 0,\\
&\text{network:} && W^{\tilde{x}}_{L} = 0, \quad b_{L} = 0, \quad W^{\bar{x}}_{L} \sim \mathcal{U}\big(-1/\sqrt{n_h}, 1/\sqrt{n_h}\big),\\
&\text{encoder:} && \theta^{b,0}_{\mathrm{enc}} \text{ from (27)},\\
&&& W^{a} \sim \mathcal{U}(-a, a), \quad \tilde{W}^{\bar{x}}_{L} = 0, \quad \tilde{b}^{\bar{x}}_{L} = 0,\\
&&& a = \sqrt{6/(n_{\bar{x}} + 6(n+1))},
\end{aligned} \quad (28)$$

where W_{L} and b_{L} are the output layer weights and bias of ϕ_{aug}, with rows $W^{\tilde{x}}_{L}$ for $f^{\mathrm{n}}_{\mathrm{aug}}$ and $W^{\bar{x}}_{L}$ for $g^{\mathrm{n}}_{\mathrm{aug}}$. n_h is the number of neurons per hidden layer, and $\tilde{W}^{\bar{x}}_{L}$ and $\tilde{b}^{\bar{x}}_{L}$ are the added-state rows of the output layer weights and bias of $\tilde{\psi}$, respectively. Here $W \sim \mathcal{U}(-a, a)$ means that every entry of W is drawn independently from the uniform distribution on $[-a, a]$. We initialise $W^{\bar{x}}_{L}$ and the hidden layers of ϕ_{aug} from $\mathcal{U}(-1/\sqrt{m}, 1/\sqrt{m})$, where m is the input size of the layer. We use Xavier initialisation for W^{a}, following [4, Eq. (31)]. We choose the uniform distribution with gain one, giving $a = \sqrt{6/(n_{\bar{x}} + 6(n+1))}$. At the start of training, $\theta_{\mathrm{base}} = \theta^{0}_{\mathrm{base}}$ and $f^{\mathrm{n}}_{\mathrm{aug}} = 0$. The augmented model and baseline model give the same $\tilde{x}$ and $\hat{y}$, given they start from the same physical state. The added states $\bar{x}$ are still updated by $g^{\mathrm{n}}_{\mathrm{aug}}$, as in [4, Eq. (29)].

E. Added-state initialisation

- Problem: the start (28) gives $\tilde{x}$ a model (f_{base}, W^{b}, output) and $\bar{x}$ none (random W^{a}, random added-state rows of ϕ_{aug}); V scores only the output and f_{aug} can lower the same error statically, so training can leave the omitted dynamics out of $\bar{x}$ (Venkatraman Sec. 2.1; Section VI tests it)
- PS is this work's, adapted from two-stage regression (Hefny Sec. 2) and its use before BPTT (Downey Sec. 4.2); it gives $\bar{x}$ a start from data, as W^{b} gives $\tilde{x}$ one from the baseline
- S1: a decoder on ψ_{a} predicts the current model's residual over τ to $\tau + H - 1$; the minimiser is the conditional mean, so $\bar{x}_{\tau|\tau}$ becomes a predictive state; trains θ_{p}, $\theta^{a}_{\mathrm{enc}}$ only, one update after every update of V
- Stopping rule: ν on calibration windows ends S1 (plateau or cap) and decides whether S2 runs
- S2: once, one-step regression of $g^{\mathrm{n}}_{\mathrm{aug}}$ on frozen encoder pairs with the recorded input, which equals the loop input when the encoder reproduces y_{k_i}; overlapping windows (?)
- Hand-over: decoder discarded, (23) continues from ϑ_{PS}; deployed model, criterion and selection unchanged; $\bar{x}$ latent; $\mathcal{M}_{\mathrm{PS}}$ named

The physical states of the baseline are initialised with the data-based fit (27) and updated through $f^{\mathrm{n}}_{\mathrm{base}}$, while the added-state rows W^{a} are randomly initialised. For the physical states their generalised coordinates q_k are directly mapped to actuator positions, $\hat{y}_k = P^{\top}q_k$ in (16). The added states are updated through the random added-state rows of ϕ_{aug}, without their initialisation or initial updates being derived from the baseline or the data. The output $\hat{y}_k = P^{\top}q_k$ contains no added state, and $\bar{x}_k$ enters $\tilde{x}_{k+1}$ only through $f^{\mathrm{n}}_{\mathrm{aug}}$ in (23). This means that for the added states V only updates their effect on the output through $f^{\mathrm{n}}_{\mathrm{aug}}$. From (23), V has no term that requires $\bar{x}$ to predict the resulting output error. This gives $f^{\mathrm{n}}_{\mathrm{aug}}$ the capability to lower part of this error with $\tilde{x}$ and $\hat{u}$ alone. Minimising V can therefore reduce the error without capturing the missing dynamics in $\bar{x}$. [**TODO:** Section VII tests whether it does.]

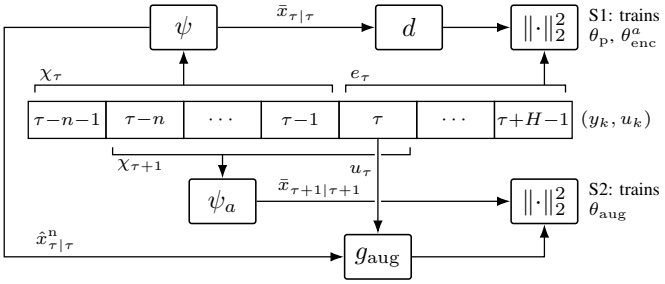


Fig. 6: Windows of PS on one record, layout adapted from [19, Fig. 2]. S1 (top): the encoder maps the window χ_τ to $\hat{x}^\mathrm{n}_{\tau|\tau}$, and the decoder d maps its added part $\bar{x}_{\tau|\tau}$ to the residuals e_τ of the next H samples (29). S2 (bottom): g_aug maps $\hat{x}^\mathrm{n}_{\tau|\tau}$ and the recorded u_τ to the added state encoded from the window shifted by one sample (32).

[**Note:** d is also a baseline parameter, need to change]

This forms a sharp contrast between the physical and added states, due to the physical states being able to directly map to the output, while also having a much better starting point.

We therefore propose predictive-state (PS) initialisation, which is a data-driven initialisation for the added states. Similar to W^b after its initialisation is done, the parameters are optimised through (23). PS is adapted from a two-stage regression of predictive-state learning [19, Sec. 2], and it makes use of initialisation before backpropagation through time(BPTT) [20, Sec. 4.2].

PS has three stages, each with its own optimisation problem. In S1, (23) is minimised as before, and after each of its updates the added-state outputs of the encoder are trained to predict the residual of the model. In S2, $g^\mathrm{n}_\mathrm{aug}$ is trained to predict the next added state. After S2, (23) is minimised from the resulting parameters. The first stage (S1) predicts the residual of the model, instead of features of future observations as in [19]. The second stage (S2) trains the nonlinear state update instead of a linear operator. We continue minimising V during the first stage, while Downey et al. finish their initialisation before starting BPTT [20, Sec. 4.2]. Fig. 6 shows which samples are used for both stages.

Stage S1 trains the encoder's added-state outputs to predict the residual of the augmented model (16) using the model parameters ϑ from the latest V update step. Each update of V is followed by one S1 update with its own Adam optimiser. For the S1 update we select a minibatch $\mathcal{B}$ of samples τ from the training data. ~~The training data is divided into excitation classes, and for classes with at least three records one is left out of S1 and S2 to evaluate ν (30). The left out data is still used in V.~~[moved to the u paragraph, M-mail] For each sample τ, the encoder uses the last $n+1$ samples up to $\tau-1$, χ_τ, to estimate the initial state $\hat{x}^\mathrm{n}_{\tau|\tau}$. From this state the augmented model is simulated in the closed loop of Section IV-B. We calculate the residual from the next $H = n+1$ samples, from τ to $\tau+H-1$. The top of Fig. 6 shows the window χ_τ and the samples of the residual e_τ. We have chosen to use the same number of samples as the encoder window. Then a temporary decoder d with only the added states as inputs $\bar{x}_{\tau|\tau} = \psi_a(\theta_\mathrm{enc}, \chi_\tau)$ is used to predict this residual:

$$
\begin{aligned}
e_\tau &= \frac{1}{\sigma_e}\left[\,(y^\mathrm{n}_\tau - \hat{y}^\mathrm{n}_{\tau|\tau})^\top \;\cdots\; (y^\mathrm{n}_{\tau+H-1} - \hat{y}^\mathrm{n}_{\tau+H-1|\tau})^\top\right]^\top, \\
d(\theta_\mathrm{p}, \bar{x}) &= W_{\mathrm{p},2}\tanh\big(W_{\mathrm{p},1}\bar{x} + b_{\mathrm{p},1}\big) + b_{\mathrm{p},2}, \\
&\min_{\theta_\mathrm{p},\,\theta^a_\mathrm{enc}} \frac{1}{|\mathcal{B}|\,H\,n_y}\sum_{\tau\in\mathcal{B}}\big\|d\big(\theta_\mathrm{p}, \psi_a(\theta_\mathrm{enc}, \chi_\tau)\big) - e_\tau\big\|_2^2,
\end{aligned}
\tag{29}
$$

where $\hat{y}^\mathrm{n}_{k|\tau}$ is the output from the closed-loop simulation of Section IV-B, starting at $\hat{x}^\mathrm{n}_{\tau|\tau}$. This simulated output is kept fixed during the S1 update. $\mathcal{B}$ is the minibatch of samples τ and σ_e is the RMS of all residual values in $\mathcal{B}$. The decoder $d : \mathbb{R}^{n_{\bar{x}}} \to \mathbb{R}^{Hn_y}$ has parameters $\theta_\mathrm{p} = (W_{\mathrm{p},1}, b_{\mathrm{p},1}, W_{\mathrm{p},2}, b_{\mathrm{p},2})$, with $W_{\mathrm{p},1} \in \mathbb{R}^{n_h \times n_{\bar{x}}}$ and $W_{\mathrm{p},2} \in \mathbb{R}^{Hn_y \times n_h}$. Both $W_{\mathrm{p},2}$ and $b_{\mathrm{p},2}$ are initially set to zero, so that the decoder starts by predicting zero residual. Only θ_p and θ^a_enc are updated by S1. The parameters of ϑ, including θ^a_enc are still trained through V.

The expected squared error is minimised by the conditional mean $\mathbb{E}[e_\tau \mid \chi_\tau]$. The decoder only sees up to $\bar{x}_{\tau|\tau}$, not the full history χ_τ, this forces the encoder to use the added states to predict the future residual. [**TODO:** Instead of using features of future observations as Hefny et al. [19, Sec. 2] for predictive-state learning, we use the model residual.] To avoid correlation between the history and future noise Hefny et al. separate the past and future samples [19, Sec. 2]. The encoder window ends at $\tau-1$, while the residual starts at τ, so no measurement sample is reused on both sides. With filtered measurement noise, this separation does not guarantee noise independence. [**TODO:** Venkatraman et al. use a similar approach by predicting future observations from the internal states during training [21, Sec. 3].] For the PS implementation the decoder predicts the model residual and is removed after S1.

[from PS, S1, M-mail] The training data is divided into excitation classes[**TODO:** see section], and for classes with at least three records one is left out of S1 and S2 to evaluate ~~ν (30)~~ S1[M-mail]. The left out data is still used in V.[end] To evaluate how well $\bar{x}_{\tau|\tau}$ performs at predicting the residual, a fixed set of windows $\mathcal{C}$ from the held out data is used. For this set the squared prediction error of the decoder divided by the squared residual is computed,

$$
\nu = \frac{\sum_{\tau\in\mathcal{C}}\big\|d\big(\theta_\mathrm{p}, \psi_a(\theta_\mathrm{enc}, \chi_\tau)\big) - e_\tau\big\|_2^2}{\sum_{\tau\in\mathcal{C}}\|e_\tau\|_2^2}. \tag{30}
$$

S1 is stopped when a maximum number of updates is reached or when ν stops improving, or at an early check when ν is above ν_max. S2 starts only if ν is below a threshold ν_max, else the decoder is removed and training continues. In both cases the encoder changes from S1 are kept. The threshold rule for S2 is given in Appendix A-E. Written as one procedure, S1 alternates the two problems: for $i = 0, 1, \ldots, i_1 - 1$,

$$
\begin{aligned}
\vartheta^{(i+1)} &= \mathcal{A}_V\big(\vartheta^{(i)}\big), \\
\big(\theta_\mathrm{p}^{(i+1)}, \theta^{a,(i+1)}_\mathrm{enc}\big) &= \mathcal{A}_{\mathrm{S1}}\big(\theta_\mathrm{p}^{(i)}, \theta^a_\mathrm{enc} \text{ of } \vartheta^{(i+1)}\big),
\end{aligned}
\tag{31}
$$

where $\mathcal{A}_V$ is one Adam step on (23), $\mathcal{A}_{\mathrm{S1}}$ one Adam step on (29), the updated θ^a_{enc} is written back into $\vartheta^{(i+1)}$, and i_1 is the update at which the stopping rule above ends S1. [**TODO:** Missing: basis of the PS switch and stop rules (S1 plateau and cap, screen update and threshold, S2 plateau and cap, snapshot sizes, calibration split as the last record of each class with three or more records). Candidate: rows of the hyperparameter table in Section VI, all HEURISTIC (`PS2Spec`; `PS2-METHOD.md` Sec. 5), noting that the S1 cap, not the plateau, ended S1 in every run so far.]

S1 has only trained the part of the encoder that initialises the added states $\bar{x}_{\tau|\tau}$, $g^{\mathrm{n}}_{\mathrm{aug}}$ is only updated by V. S2 trains the state update itself. In S2 $g^{\mathrm{n}}_{\mathrm{aug}}$ is trained to predict the next added state $\bar{x}_{k+1}$ from the full state $\hat{x}^{\mathrm{n}}_k$ and the input u^{n}_k. For S2 samples k_i, $i = 1, \ldots, N_{\mathcal{P}}$, from the training data are selected, except the same held-out records from S1 are left out. For each sample the encoder estimates $\hat{x}^{\mathrm{n}}_{k_i|k_i}$ from χ_{k_i} and $\bar{x}_{k_i+1|k_i+1}$ from the next window χ_{k_i+1}. Fig. 6 shows an example with $k_i = \tau$. The window χ_τ at the top is used for $\hat{x}^{\mathrm{n}}_{k_i|k_i}$, and the shifted window at the bottom is used for $\bar{x}_{k_i+1|k_i+1}$. We calculate these pairs once after S1, this set $\mathcal{P}$in (32) is kept fixed.

During S2 the model is not simulated in closed-loop and V is paused. We instead train $g^{\mathrm{n}}_{\mathrm{aug}}$ on $\mathcal{P}$ using one-step regression with the recorded actuator force $u^{\mathrm{n}}_{k_i}$ as input, and a separate Adam optimiser:

$$\mathcal{P} = \left\{\left(\hat{x}^{\mathrm{n}}_{k_i|k_i},\, u^{\mathrm{n}}_{k_i},\, \bar{x}_{k_i+1|k_i+1}\right)\right\}_{i=1}^{N_{\mathcal{P}}},$$
$$\hat{x}^{\mathrm{n}}_{k_i|k_i} = \psi(\theta_{\mathrm{enc}}, \chi_{k_i}), \quad \bar{x}_{k_i+1|k_i+1} = \psi_a(\theta_{\mathrm{enc}}, \chi_{k_i+1}),$$
$$\min_{\theta_{\mathrm{aug}}} \frac{\sum_{i\in\mathcal{B}_{\mathcal{P}}} \left\| g^{\mathrm{n}}_{\mathrm{aug}}\left(\theta_{\mathrm{aug}}, \hat{x}^{\mathrm{n}}_{k_i|k_i}, u^{\mathrm{n}}_{k_i}\right) - \bar{x}_{k_i+1|k_i+1} \right\|_2^2}{\sum_{i\in\mathcal{B}_{\mathcal{P}}} \sum_{j=1}^{n_{\bar{x}}} \left(\bar{x}_{j,k_i+1|k_i+1} - m_j\right)^2}, \tag{32}$$

where θ_{enc} is not updated during S2, $\mathcal{B}_{\mathcal{P}} \subset \mathcal{P}$ is a minibatch, $\bar{x}_{j,\cdot}$ is the jth added state and m_j is the mean of $\bar{x}_{j,k_i+1|k_i+1}$ over $\mathcal{B}_{\mathcal{P}}$. The cost is the fraction of the variance in the next added states that is not explained by the prediction.

The parameters with the lowest cost on the separate set of pairs from the held-out records are kept. For S2 we use the recorded actuator force $u^{\mathrm{n}}_{k_i}$ applied to the system at k_i. The encoder estimates $\bar{x}_{k_i+1|k_i+1}$ from χ_{k_i+1}, using the data up to $k_i + 1$. At the start of a window in (21) $x^c = 0$, resulting in the model input

$$\hat{u}_{\mathrm{act},k_i} = u_{\mathrm{act},k_i} + D_{\mathrm{fb}}\left(y_{k_i} - \hat{y}_{k_i|k_i}\right),$$

when the encoder reproduces y_{k_i} this equals the recorded input, because at this sample only the controller feedthrough D_{fb} acts on the output error. For S2 $g^{\mathrm{n}}_{\mathrm{aug}}$ is therefore trained with approximately the input it receives at that sample in the closed-loop simulation of V. S2 updates all parameters θ_{aug} of ϕ_{aug}. The cost only depends on the added-state rows, so the physical rows of W_L and b_L are not changed. The hidden layers of ϕ_{aug} are shared by $f^{\mathrm{n}}_{\mathrm{aug}}$ and $g^{\mathrm{n}}_{\mathrm{aug}}$, so S2 also updates $f^{\mathrm{n}}_{\mathrm{aug}}$. [**TODO:** The encoder windows at k_i and $k_i + 1$ share measurement samples, so noise enters both sides of the S2 regression. Hefny et al. discuss how correlated noise can bias such a fit [19, Sec. 2]. We do not correct for this effect in S2.]

When S2 is completed the optimisation continues minimising (23), starting from

$$\vartheta_{\mathrm{PS}} = \left(\beta^{(i_1)},\, \theta^{\mathrm{S2}}_{\mathrm{aug}},\, \theta^{(i_1)}_{\mathrm{enc}}\right), \tag{33}$$

followed by the L-BFGS polish and selection of Section IV-C. Here $\beta^{(i_1)}$ and $\theta^{(i_1)}_{\mathrm{enc}}$ are the parameters after S1 and $\theta^{\mathrm{S2}}_{\mathrm{aug}}$ contains the parameters after S2. When S2 is skipped, ϑ_{PS} contains the parameters at the end of S1.

PS has only changed the starting point of minimising V with its initialisation phases, not what is being minimised. The models with and without PS have the same number of parameters, since the decoder is removed.

V. Interpretability by Orthogonal Construction

- Goal from Section III: an estimate of θ_{base} that is unique under stated conditions; names IV-A and IV-B (style profile)
- Contribution: the extension Györök 2026 Sec. 6 leaves open: a state-space baseline without full-state measurement, here nonlinear in its parameters and identified in closed loop (this work)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

The parameter pair $(\theta_{\mathrm{base}}, \theta_{\mathrm{aug}})$ minimising the loss function of Section IV-C can be non-unique: different pairs can give the same fit [7, Sec. 3.1]. Meaning that the rigid-body dynamics that the baseline parameters θ_{base} are able to explain from the data, can instead be represented by a neural network correction. To prevent the neural network from correcting what θ_{base} can represent we introduce orthogonality by construction [7]. Orthogonality here is defined as the neural network correction to the physical states having no component in the directions in which a change of θ_{base} is able to change the next state predicted by the baseline. This gives us a unique estimate of θ_{base} up to first order, but it does not guarantee recovering the true baseline parameters.

Györök et al. have constructed this for input-output baseline models that are linear in their parameters [7]. We extend this to state-space models nonlinear in their parameters, specifically for our gantry system.

A. Orthogonal-by-construction parametrisation

- Györök's construction as adopted: baseline linear in its parameters, regressor at measured input-output data (past outputs, past and current inputs), stacked over the training data; lower case per sample, upper case stacked (2026 Eqs. (1), (2), (5))
- θ_{aux} is the least-squares fit of the network output by the regressor; its fitted part is removed and the remainder is orthogonal to Φ on the training data (2026 Eqs. (8) to (11), Lemma 3)

- θ_{aux} exists for full column rank of Φ: identifiable baseline, weakly persistently exciting input (2026 Eqs. (6), (7), Assumption 1)
- Data-driven: orthogonality holds on the data used, the projection depends on their distribution; any fixed set may be used, the analysis uses the whole training set (2026 Sec. 3.2, text after Eqs. (12), (13))
- Uniqueness of $\hat{\theta}_{\text{b}}$ shown under Assumption 1 and exact reproduction of the noise-free dynamics on the same training data (2026 Sec. 4.2, Assumption 6)

For a baseline linear in its parameters, $\hat{y}_k = \varphi(x_k)\theta_{\text{b}}$, changing θ_{b} changes the model output $\hat{y}_k$ by a linear combination of the columns of the regressor $\varphi(x_k)$ [7, Eqs. (2), (39)]. Györök et al. subtract from the neural network output its least-squares fit by the stacked regressor [7, Eqs. (8), (9), (11), Lemma 3]:

$$\begin{aligned} \hat{Y} &= \Phi\theta_{\text{b}} + \tilde{F}^{\text{ANN}}, \\ \tilde{F}^{\text{ANN}} &= F^{\text{ANN}} - \Phi\theta_{\text{aux}}, \\ \theta_{\text{aux}} &= \left(\Phi^\top\Phi\right)^{-1}\Phi^\top F^{\text{ANN}}, \\ 0 &= \Phi^\top \tilde{F}^{\text{ANN}}, \end{aligned} \tag{34}$$

where x_k is the vector of past measured outputs and past and current inputs [7, Eq. (1)], $\hat{Y}$ stacks the predicted outputs over the training data, Φ is the stacked regressor, F^{ANN} stacks the parallel network outputs [7, Eq. (5)] and $\tilde{F}^{\text{ANN}}$ is the projected network output [7, Eqs. (9), (10)].

The projected network output $\tilde{F}^{\text{ANN}}$ is orthogonal to the regressor [7, Lemma 3]. Calculating θ_{aux} requires $(\Phi^\top\Phi)^{-1}$, which exists only if Φ has full column rank [7, Assumption 1]. Therefore different values of θ_{b} are unable to give the same stacked prediction on the training data. Györök et al. call an input with this property persistently exciting [7, Eq. (6), Sec. 4.3]. This also requires an identifiable baseline, where different values of θ_{b} give different functions $\varphi(\cdot)\theta_{\text{b}}$.

B. Extension to the gantry

- Three differences, each changing one element of (34): regressor, data, prediction (this work)
- Regressor: Jacobian of the normalised transition in β at β_{lin}, since the baseline is nonlinear through $M(Y)^{-1}$; β because only the ten combinations are identifiable (adapted from Györök 2025 Sec. 4, Eq. (15); 2026 Assumption 1; Section II-C; D-190, D-229); offset column Γ included (D-257)
- Data: reference tuples reconstructed from the measured positions, regressor at the data as in 2026 Eq. (5) rather than at a simulated baseline (2025 Sec. 3); each with a successor; built once before training (D-252, D-254)
- Projection on the stacked tuples, with the orthogonality of the section opening as its last line; holds for the stack at $\bar{x} = 0$ (adapted from 2026 Eqs. (8) to (11), Lemma 3)
- Prediction: the correction at every step of the closed-loop state update, physical rows only; coefficient differentiated (2026 Eqs. (9), (10), (13); added states this work)

Our baseline violates three requirements of the projection in (34). Its state update is nonlinear in the parameters, because the next state update depends on $M(Y)^{-1}$ in (17), violating the linear requirement for the regressor. Instead of the regressor the Jacobian of the baseline state update is taken with respect to β, as in the orthogonal regularisation method of Györök et al. [22, Sec. 4, Eq. (15)]. Like the regressor, the stacked Jacobian must have full column rank. This requires identifiable parameters, which is why we use β. The Jacobian is taken at a linearisation point β_{lin} as

$$\varphi(\beta_{\text{lin}}, \tilde{x}^{\text{n}}, u^{\text{n}}) = \left.\frac{\partial f^{\text{n}}_{\text{base}}\left(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}, u^{\text{n}}\right)}{\partial\beta}\right|_{\beta=\beta_{\text{lin}}}, \tag{35}$$

where $\beta_{\text{lin}} \in \mathbb{R}^{10}$ and $\varphi : \mathbb{R}^{10} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^{6\times 10}$. The linearisation also leaves an offset that does not depend on β [22, Eq. (16)]:

$$\begin{aligned} f^{\text{n}}_{\text{base}}\left(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}, u^{\text{n}}\right) &\approx \varphi\,\beta + \gamma, \\ \gamma &= f^{\text{n}}_{\text{base}}\left(\theta_{\text{base}}(\beta_{\text{lin}}), \tilde{x}^{\text{n}}, u^{\text{n}}\right) - \varphi\,\beta_{\text{lin}}, \end{aligned} \tag{36}$$

where $\gamma : \mathbb{R}^{10} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^6$, so that $\varphi\beta_{\text{lin}} + \gamma$ is the next state predicted by the baseline at β_{lin}. Györök et al. add this offset as an extra column to the Jacobian, to include all components of the baseline model in the orthogonality [22, Eq. (18)]. We include it as well, so the neural network has no component along the stacked next states predicted by the baseline. ~~The offset does not change the first-order estimate (41).~~[moved to V-C, M-mail] ~~It~~ The offset[M-mail] does not fully prevent negation [9, Sec. 5.3.1.3], where the neural network cancels part of the baseline response, because only the part of the network along the stacked baseline response is excluded.

The Jacobian also depends on the state, while the regressor of Györök et al. uses only measured inputs and outputs [7, Eqs. (1), (2)]. Since only the positions are measured, we evaluate the Jacobian at these positions with the velocities reconstructed from them:

$$\begin{aligned} \tilde{x}_i &= \operatorname{col}(q_i, \dot{q}_i), \qquad q_i = P^{-\top} y_i, \\ \dot{q}_i &= \frac{-q_{i+2} + 8q_{i+1} - 8q_{i-1} + q_{i-2}}{12\,T_s}, \end{aligned} \tag{37}$$

where y_i is the measured actuator position at sample i. Each tuple $(\tilde{x}_i, \bar{x}_i, u_i)$ uses the recorded actuator force u_i and $\bar{x}_i = 0$, with the physical state and input normalised to $\tilde{x}^{\text{n}}_i$ and u^{n}_i using (18). The tuples are calculated once before training, using all samples for which (37) can be calculated at i and at the next sample $i+1$.

Stacking the Jacobian, the offset and the network output over the tuples, the projection of (34) becomes

$$\begin{aligned} \Phi_\Gamma &= \operatorname{col}\left(\begin{bmatrix}\varphi & \gamma\end{bmatrix}(\beta_{\text{lin}}, \tilde{x}^{\text{n}}_i, u^{\text{n}}_i)\right)_{i\in\mathcal{I}}, \\ F^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) &= \operatorname{col}\left(f^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}, \operatorname{col}(\tilde{x}^{\text{n}}_i, 0), u^{\text{n}}_i)\right)_{i\in\mathcal{I}}, \\ \theta_{\text{aux}}(\theta_{\text{aug}}) &= \left(\Phi_\Gamma^\top\Phi_\Gamma\right)^{-1}\Phi_\Gamma^\top F^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}), \\ \tilde{F}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) &= F^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}) - \Phi_\Gamma\,\theta_{\text{aux}}(\theta_{\text{aug}}), \\ 0 &= \Phi_\Gamma^\top \tilde{F}^{\text{n}}_{\text{aug}}(\theta_{\text{aug}}), \end{aligned} \tag{38}$$

where $\mathcal{I}$ is the set of tuples, $N = |\mathcal{I}|$, $\Phi_\Gamma = [\Phi\ \Gamma] \in \mathbb{R}^{6N\times 11}$ stacks φ and γ evaluated at β_{lin}, $F^{\text{n}}_{\text{aug}}, \tilde{F}^{\text{n}}_{\text{aug}} \in \mathbb{R}^{6N}$ and

$\theta_{\text{aux}} \in \mathbb{R}^{11}$. The projected network output $\tilde{F}^{\text{n}}_{\text{aug}}$ therefore has no component along any column of Φ_Γ, which satisfies our orthogonality definition at the start of this section. This holds for the stacked tuples at $\bar{x} = 0$, not for each tuple separately. We use a state-space model predicting the next state instead of the output. The fitted part is removed from the neural network correction of the physical state at every step in the closed-loop simulation. At each step, we subtract the Jacobian and offset multiplied by θ_{aux} from the network correction, evaluated at the current state and input. Györök et al. apply the same correction to the predicted output [7, Eqs. (9), (13)]. The baseline has no added states, so its Jacobian and offset are zero in the added-state rows and $g^{\text{n}}_{\text{aug}}$ is orthogonal to it by definition. We therefore only correct the physical rows:

$$\begin{aligned} f^{\text{n}}_{\text{obc}}(\theta_{\text{aug}}, \tilde{x}^{\text{n}}, u^{\text{n}}) &= [\varphi \;\; \gamma](\beta_{\text{lin}}, \tilde{x}^{\text{n}}, u^{\text{n}})\, \theta_{\text{aux}}(\theta_{\text{aug}}), \\ \tilde{x}^{\text{n}}_{k+1|\tau} &= f^{\text{n}}_{\text{base}} + f^{\text{n}}_{\text{aug}} - f^{\text{n}}_{\text{obc}}, \\ \bar{x}_{k+1|\tau} &= g^{\text{n}}_{\text{aug}}, \end{aligned} \tag{39}$$

where $f^{\text{n}}_{\text{obc}} : \mathbb{R}^{n_{\text{aug}}} \times \mathbb{R}^6 \times \mathbb{R}^3 \to \mathbb{R}^6$ is evaluated at $(\tilde{x}^{\text{n}}_{k|\tau}, \hat{u}^{\text{n}}_{k|\tau})$, and $f^{\text{n}}_{\text{base}}$, $f^{\text{n}}_{\text{aug}}$ and $g^{\text{n}}_{\text{aug}}$ take the arguments of (23). We differentiate θ_{aux} with respect to θ_{aug}, as in [7, Eqs. (9), (10)], so the network is trained using the gradient of its projected output.

C. Identification and recovery

- $\mathcal{M}_{\text{OBC}}$: the augmented model of Section III with the PS start and the correction; identified by (23) with (39) as the physical-state update, everything of Section III-C reused (this work)
- Rules, each from a dependence: θ_{aux} re-solved at every evaluation; β_{lin} per epoch, a fixed objective per epoch while the point follows the estimate; 2025 Sec. 4 options not taken (cost, detuned start), Remark 2 adapted (D-190, D-254); rebuild after restore, polish compared with the rebuilt model (D-253, revnote)
- Recovery: Ξ (40), the unmodelled terms of 2026 Eqs. (14), (15) plus reconstruction error and noise; the one-step error and its two first-order minimisers (41); form of 2026 Eq. (21) without the exact fit of Assumption 6; true iff $\Phi^\top \Xi = 0$ (Condition 4, Remark 5); Γ does not enter (D-257)
- Limits: a one-step surrogate at $\bar{x} = 0$, not the closed-loop optimum; negation is treated in IV-B
- Measures: cos, ρ, Π (42) (supervisor: orthogonality is measured by the inner product); negation by the cosine with the baseline's one-step increment (D-257); c_j dropped (correlated columns of Φ; the per-parameter bias is (41))

The augmented model is identified as in Section IV-C, with the added-state initialisation of Section IV-E. With orthogonality by construction the state update in (23) is instead the projected state update (39). Since θ_{aux} depends on θ_{aug}, we solve (38) at every evaluation of V. At the start of each epoch β_{lin} is set to the current β, and it is held fixed during the epoch. Györök et al. update β_{lin} at every iteration, which requires rebuilding Φ_Γ at each evaluation of V [22, Sec. 4]. The other proposed method is to keep β_{lin} fixed at the nominal parameters, assuming these are close to the true parameters [22, Sec. 4]. ~~Training as stated in Section VI starts~~ Since training may start[M-mail] from detuned parameters, we instead adapt their recomputation at the start of each epoch [22, Remark 2]. We load the best checkpoint and rebuild Φ_Γ using its β. The polish minimises the same projected objective, with Φ_Γ held fixed at this β. The polish is only kept if its validation error is lower. [**TODO:** Clarify (D-274): "lower" means lower than the validation error of the best checkpoint; if the polish is not kept, the model with the rebuilt Φ_Γ is kept, whose validation error can differ slightly from that of the checkpoint (run 229: 2.926741×10^{-1} vs 2.926736×10^{-1}).]

The projection does not guarantee that the true parameters $\beta^\star \in \mathbb{R}^{10}$ are recovered. That depends on whether what the baseline misses on the tuples is itself orthogonal to the stacked Jacobian:

$$\Xi = \operatorname{col}\big(\tilde{x}^{\text{n}}_{i+1} - f^{\text{n}}_{\text{base}}(\theta_{\text{base}}(\beta^\star), \tilde{x}^{\text{n}}_i, u^{\text{n}}_i)\big)_{i \in \mathcal{I}}, \tag{40}$$

where $\Xi \in \mathbb{R}^{6N}$ contains the unmodelled terms of [7, Eqs. (14), (15)], the reconstruction error of (37) and the measurement noise. To first order around $\beta^\star$, the one-step error on the tuples is $\Xi - \Phi(\beta - \beta^\star) - \tilde{F}^{\text{n}}_{\text{aug}}$. Minimising the squared one-step error over β gives $\hat{\beta}$. Without the projection, $F^{\text{n}}_{\text{aug}}$ replaces $\tilde{F}^{\text{n}}_{\text{aug}}$, which gives $\hat{\beta}_{\text{np}}$:

$$\begin{aligned} \hat{\beta} &= \beta^\star + \big(\Phi^\top \Phi\big)^{-1} \Phi^\top \Xi, \\ \hat{\beta}_{\text{np}} &= \beta^\star + \big(\Phi^\top \Phi\big)^{-1} \Phi^\top \big(\Xi - F^{\text{n}}_{\text{aug}}\big). \end{aligned} \tag{41}$$

With the projection, $\hat{\beta}$ does not depend on the network output, since $\Phi^\top \tilde{F}^{\text{n}}_{\text{aug}} = 0$ by (38). [from V-B, M-mail] The offset does not change the first-order estimate (41).[end] Without the projection, $\Phi^\top F^{\text{n}}_{\text{aug}}$ is still in the estimate $\hat{\beta}_{\text{np}}$. The expression for $\hat{\beta} - \beta^\star$ has the same form as Györök et al. [7, Eq. (21)], which they derived assuming an exact fit of noise-free data [7, Assumption 6]. We obtain it by minimising the squared one-step error. The estimate equals $\beta^\star$ if and only if $\Phi^\top \Xi = 0$, requiring Ξ to be orthogonal to the columns of Φ on the tuples [7, Condition 4, Remark 5]. This is a first-order result for the one-step least-squares problem on the fixed reference slice $\bar{x} = 0$; it does not predict the parameter bias from minimising the closed-loop loss (23) with active added states.

We measure orthogonality using the normalised inner product, with the physical state normalised using (18):

$$\begin{aligned} \cos \angle(a, b) &= \frac{\langle a, b \rangle}{\|a\|_2 \, \|b\|_2}, \\ \rho(d) &= \max_{v \in \operatorname{range} \Phi \setminus \{0\}} \cos \angle(v, d) = \frac{\|\Pi d\|_2}{\|d\|_2}, \\ \Pi &= \Phi\big(\Phi^\top \Phi\big)^{-1} \Phi^\top, \\ c_{\text{neg}} &= \cos \angle\Big(F^{\text{n}}_{\text{aug}}, \operatorname{col}\big(f^{\text{n}}_{\text{base}}(\theta_{\text{base}}(\beta), \tilde{x}^{\text{n}}_i, u^{\text{n}}_i) - \tilde{x}^{\text{n}}_i\big)_{i \in \mathcal{I}}\Big), \end{aligned} \tag{42}$$

where $a, b, d \in \mathbb{R}^{6N}$ are nonzero and Π is the orthogonal projector onto $\operatorname{range} \Phi$. $\rho(d)$ gives the largest normalised inner product between d and a vector in $\operatorname{range} \Phi$. It is zero if and

only if $\Phi^\top d = 0$. Negation is measured by c_{neg}, which is the cosine between the network output and the one-step state change predicted by the baseline.

[**Note:** Lets use something else then d.]

VI. Experiment design

- Opening: the records are simulated on a model of a confidential industrial gantry; quantities are given relative; design details and checks in Appendix D

The system used to obtain the data is simulated. When referring to the system in this section, we refer to this simulated model. The data consist of simulated records, with motion profiles, limits and noise level taken from an industrial gantry whose data is confidential. Frequencies are therefore given relative to the controller bandwidth f_b, positions relative to the training range, and velocities and accelerations relative to the machine limits. The design details and their checks are in Appendix D.

A. Simulated system

- The data is simulated: the system is the baseline plus rail friction and a payload absorber, both absent from the baseline
- The friction subtracts from the actuator forces; a smoothed Coulomb law, Coulomb-like on the excited records
- The absorber adds one coordinate, coupled through the inertia; matrices written out as in Section II
- Parameter values with their basis; both absorber features lie above the crossover of the loop
- One known controller K_1, the same in every record; the injected force is the only feedforward
- Noise matched to the machine; noise-free twin of every record; SNR (?: confidentiality)

The system consists of the baseline model (4), with two additions. The first is Coulomb friction at the actuators X_1, X_2 and Y, as modelled by García-Herreros et al. [1]. We deliberately left it out of the baseline, to add nonlinear unmodelled dynamics to the system. The friction works as a force against each actuator force, this is modelled as a term entering (4) through the input. For the second addition, we split the payload mass m_h into m_a and $m_{h,r} = m_h - m_a$, with m_a connected to $m_{h,r}$ by a spring and damper in the Y direction (Fig. 7). This adds a vibration mode which the baseline model is not able to represent with its states. With the friction acting against each actuator force on the input, (4) becomes

$$\begin{aligned} M(Y)\ddot{q} + C\dot{q} + Kq &= P\big(u_{\text{act}} - F_{\text{fr}}(P^\top \dot{q})\big), \\ F_{\text{fr},i}(v_i) &= c_{c,i}\tanh\big(v_i/v_{\text{fr}}\big), \end{aligned} \tag{43a}$$

where $P^\top \dot{q}$ gives the actuator velocities for $i \in \{X_1, X_2, Y\}$, and v_{fr} sets the velocity scale of the smoothing. For $|v_i| \gg v_{\text{fr}}$, the friction approaches the Coulomb law $F_{\text{fr},i} = c_{c,i}\,\text{sign}(v_i)$ of [1]. $v_{\text{fr}} = 1$ mm/s was chosen by simulating the loop with values between 0.1 and 10 mm/s. Higher values turn the friction into viscous damping under the multisine, and lower values keep the standstill servo error above the noise level.

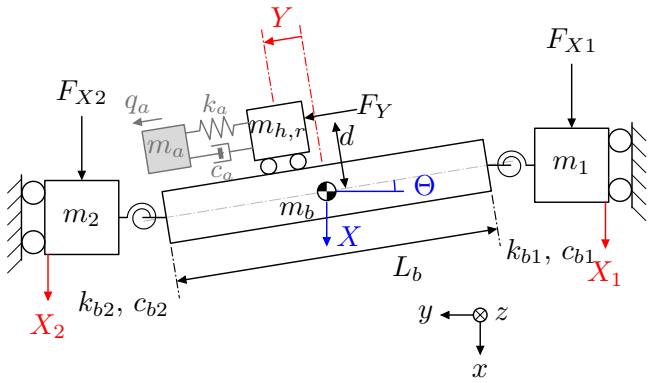


Fig. 7: [**TODO:** Simulated gantry with a flexibly mounted part m_a of the payload mass m_h; the rest of the payload is $m_{h,r} = m_h - m_a$. The light-grey elements (m_a, k_a, c_a, q_a) are absent from the baseline.]

The added mass adds the displacement q_a of m_a relative to $m_{h,r}$ along Y. With $q_{\text{e}} = \begin{bmatrix} q^\top & q_a \end{bmatrix}^\top$, the system is

$$M_{\text{e}}(Y, q_a)\,\ddot{q}_{\text{e}} + C_{\text{e}}\dot{q}_{\text{e}} + K_{\text{e}}q_{\text{e}} = \begin{bmatrix} P\big(u_{\text{act}} - F_{\text{fr}}(P^\top \dot{q})\big) \\ 0 \end{bmatrix}, \tag{43b}$$

with

$$M_{\text{e}}(Y, q_a) = \begin{bmatrix} M(Y) + M_a(Y,\ell) & m_a \begin{bmatrix} 0 & -d & 1 \end{bmatrix}^\top \\ m_a \begin{bmatrix} 0 & -d & 1 \end{bmatrix} & m_a \end{bmatrix},$$

$$M_a(Y,\ell) = m_a\ell \begin{bmatrix} 0 & -1 & 0 \\ -1 & 2Y+\ell & 0 \\ 0 & 0 & 0 \end{bmatrix},$$

$$C_{\text{e}} = \text{diag}(C, c_a), \quad K_{\text{e}} = \text{diag}(K, k_a),$$

where $M(Y)$ is (5) with m_h the total payload mass. $M_a(Y,\ell)$ corrects it for m_a being at $Y + \ell$ instead of Y, where $\ell = L_0 + q_a$ and L_0 is the offset of the added mass along Y. $k_a = m_a(2\pi f_a)^2$ and $c_a = 2\zeta_a\sqrt{k_a m_a}$, with natural frequency f_a and damping ratio ζ_a. The added mass changes the inertia through M_a, which also includes its coupling to Θ and Y. We simulate the system with fixed-step RK4 at $f_{\text{s}} = 20$ kHz. All parameter values are listed in Table IV. The friction levels c_c are taken from García-Herreros et al. [1, Table I]. The added mass creates an anti-resonance at f_a and a resonance at $f_a\sqrt{1 + m_a/m_{h,r}} \approx 2.12\,f_b$, in the transfer function from F_Y to Y. Both are designed to be above the crossover of the loop with K_1 (Section VI-B). As a result, the feedback does not suppress them, and we can identify them from the data. All data is generated in closed loop with the controller K_1 of Appendix D-A, which is the known controller of Section IV-B. The only feedforward, u^{ff} in (20), is the injected excitation force. Independent coloured noise is added to the position measurements of each axis, $y = q_{\text{act}} + v$. The simulated servo error matches at standstill the noise level and spectral shape of the industrial gantry, but its resonances are left out (Appendix D-A). The servo-error signal-to-noise ratio (SNR) ranges from 58 to 74 dB on the X actuators and 71 to 80 dB on Y, depending on the motion of the record. The data is also generated without measurement noise, with output y°. [**TODO:**

dB in combination with RMS migth be sensitive information.] [**TODO:** Missing: what may be named about the machine and its noise. Candidate: ask Jasper or Dragan; if allowed, list $f_{v,j}$, λ_j, $\sigma_{v,j}$ in Appendix D-A.]

B. Records

- A record: 12 s in closed loop with K_1, one of five motion types, a multisine added except on the operational moves; noisy and noise-free
- The motion profiles excite up to the crossover above the noise; the multisine band starts at the highest crossover, ends at 90 % of the model difference and holds the absorber features of the simulated system (simulation only)
- Split 18 / 6 / 11; validation only selects; results on all three sets, comparisons on the test records
- Test records by layer: interpolation, operational motion, extrapolation in one setting, stress record E6, multisine twins for evaluation only
- Model rate, anti-alias filter and window checked on the simulated loop (appendix)

The data is divided into five types of motion profiles and each record is 12 s (Appendix D-B). The motion profiles excite the low frequencies up till the loop crossover, for at least 59 dB above the noise. A multisine force is added to each of the generalised channels X, Θ, Y, except in records with an operational motion profile. The force is applied to the actuators through P^{-1}. The multisine band starts at the highest loop crossover $1.06\, f_b$, to prevent feedback suppression. The band ends at $2.97\, f_b$, which keeps 90 % of the squared frequency-response error of the baseline above the crossover. This was measured on the system without noise. The band contains the anti-resonance and the resonance of the added mass (Fig. 14), for all the positions used in training. This was determined from the frequency response of the known system model at each training position.

We use 18 records for training, 6 for validation and 11 for testing (Table X in Appendix D-B). Seven of the test records without multisine are also evaluated with the training multisine added. Training and validation include all five motion types, with $|Y| \leq Y_{\mathrm{tr}}$, up to 75 % of the acceleration limit and 75 % of the velocity limit. The validation records are used to select the best-performing checkpoints and the hyperparameters for the black box. We separate the test records into interpolation and extrapolation. Interpolation stays within the training range, while each extrapolation record exceeds one of the limits on Y, acceleration or velocity. The results for all three sets are reported, but performance is evaluated on the test set. [**TODO:** Missing: whether E6 (a measured machine move) and the relative limits may be described this way. Candidate: ask Jasper or Dragan.]

The models run at $T_s = 0.25$ ms on training windows of 0.1 s. The rate, its anti-alias filter and the window length are supported by the checks in Table XI.

TABLE I: Compared models. Start: the ten combinations at the start of training; final model: $\mathcal{M}_{\mathrm{OBC}}$.

Model	Added states	OBC	Start
$\mathcal{M}_{\mathrm{LPV}}$	0	no	true, untrained
$\mathcal{M}_{\mathrm{U}}$	2	no	10 % detuned
$\mathcal{M}_{\mathrm{OBC}}$	2	yes	10 % detuned
$\mathcal{M}_{\mathrm{BB}}$	black box	no	no physics

C. Compared models

- Each comparison answers one aspect of the research question
- Grey-box models: same records, settings, budget and checkpoint rule; PS start of the added states; 10 % detuned start of the combinations; one seed
- The runs that justify $n_{\bar{x}}$, PS and the start are in the appendix (?: label)
- Black box: SUBNET of the closed loop, no physics, no controller, sized on the validation records

The models of Table I are compared in pairs, each answering one aspect of the research question. The pairs are the final model $\mathcal{M}_{\mathrm{OBC}}$ with the baseline $\mathcal{M}_{\mathrm{LPV}}$ and the black box $\mathcal{M}_{\mathrm{BB}}$ for the augmentation, and $\mathcal{M}_{\mathrm{OBC}}$ with $\mathcal{M}_{\mathrm{U}}$ for the orthogonal construction. The extrapolation records compare the final model with the black box (Aspect 4). $\mathcal{M}_{\mathrm{LPV}}$ is the baseline at the true parameters. [**TODO:** Missing: model notation. Candidate: the calligraphic names of the table (README open conflict on model names; same `\todo` as in Section V).] We identify $\mathcal{M}_{\mathrm{U}}$ and $\mathcal{M}_{\mathrm{OBC}}$ by minimising the loss of Section IV-C with the same training setup (Table XII). This includes PS to initialise the added states (Section IV-E). The parameter combinations θ_{base} start 10 % larger or smaller in magnitude than their true values, and every run uses the same signs. Appendix C shows the runs used to choose $n_{\bar{x}}$ and the PS initialisation of the added states. [**TODO:** Missing: the label of the appendix with the choice-justifying runs, and whether they are compared on the validation records. Candidate: a new appendix section on the validation records (THESIS-RESULTS, recommendation of 2026-10-16, not yet confirmed).] The black box $\mathcal{M}_{\mathrm{BB}}$ is a SUBNET model [16] that predicts the closed-loop servo error from the reference and injected force, without modelling the controller separately (Appendix D-D). Its state count, network widths and learning rate are selected from 11 configurations using the validation records, with the same data and training and encoder window as the grey-box models.

D. Metrics

- Every model is simulated over whole test records: grey-box models in closed loop with K_1, the black box in free run
- Primary metric: the simulation error relative to the servo error, NRMS_e, per stage axis; beside it the RMS in metres and the noise floor
- Parameter error per combination against the true values; m_Δ scaled by the mean actuator mass; simulation only

- The two predictions of Section V: the network stays out of the parameter directions, and the remaining parameter error is the first-order bias

All models are evaluated based on its output simulated over the full records. Since the reference is known, we compare the predicted servo error with the recorded servo error $y - r$. The primary metric is the NRMS of [16, Eq. (43)], adapted by using the servo error as output. We also report the RMS in metres and the noise floor:

$$\begin{aligned} \mathrm{RMS}_j &= \Big(\frac{1}{N-n-1}\sum_{k=n+1}^{N-1}(\hat{y}_{j,k}-y_{j,k})^2\Big)^{1/2}, \\ \mathrm{NRMS}_{e,j} &= \frac{\mathrm{RMS}_j}{\sigma_{e,j}}, \\ \mathrm{RMS}_j^{\mathrm{fl}} &= \Big(\frac{1}{N-n-1}\sum_{k=n+1}^{N-1}(y_{j,k}-y^{\circ}_{j,k})^2\Big)^{1/2}, \end{aligned} \tag{44}$$

where j is the stage axis, N the record length, $n+1$ the encoder window length and $\sigma_{e,j}$ the population standard deviation of the servo error $y_j - r_j$ over the same samples. An NRMS of 100 % means the model performs no better than predicting the mean servo error. The noise floor is the RMS difference between the measured and noise-free outputs.

The parameter error of combination j is

$$\begin{aligned} \delta_j &= \frac{\theta_{\mathrm{base},j} - \theta^{\star}_{\mathrm{base},j}}{s_j}, \\ s_j &= \begin{cases} (m_1+m_2)/2, & \text{for } m_\Delta, \\ |\theta^{\star}_{\mathrm{base},j}|, & \text{otherwise,} \end{cases} \end{aligned} \tag{45}$$

where the true value of $m_\Delta = m_1 - m_2$ is close to zero, so we scale its error by the mean actuator mass $(m_1+m_2)/2$ instead.

For the orthogonal construction of Section V, the overlap ρ of (42) is computed at $\bar{x} = 0$ on the one-step tuples of the training, validation and test records (test: I1 to I6), for the network of $\mathcal{M}_{\mathrm{U}}$, the corrected network of $\mathcal{M}_{\mathrm{OBC}}$ and the missing dynamics Ξ. On the training tuples the corrected overlap is zero by construction.

VII. Results

- Test records, NRMS_e with the noise floor; training and validation in the appendix (D-269)

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

All errors are reported on the test set in NRMS_e of (44), against the noise floor. Appendix B gives the training and validation records.

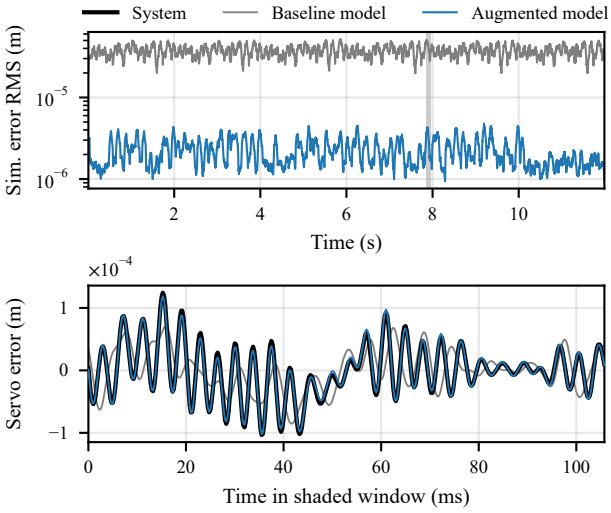
A. Augmentation structure

- Final model against the baseline and the black box, with and without multisine (Table II, Fig. 8)
- The final model has the resonance of the system, the baseline none (Fig. 9)

TABLE II: NRMS_e per axis, mean over test records I1 to I6, with and without multisine.

	Multisine			No multisine		
NRMS_e (%)	X_1	X_2	Y	X_1	X_2	Y
Baseline, true θ	18.8	20.7	116.6	4.5	4.6	3.4
Augmented (OBC), detuned $\theta^{\dagger}$	9.7	9.5	6.2	4.7	4.8	3.8
Black-box model	47.8	47.8	35.9	104.8	97.0	127.7
Noise floor	0.06	0.06	0.01	0.04	0.04	0.01

† preliminary, update 19500 of 33400



augmented model (R1): preliminary, update 19500 of 33400

Fig. 8: Test record I1, Y axis. Top: moving RMS of the simulation error. Bottom: servo error in the shaded window.

- PS start and added states compared on validation in the appendix

Table II compares the final grey box model with the baseline at the true parameters and with the black box (Aspect 1). For the data with multisine, the final model lowers the error on Y by more than an order of magnitude and halves the error on X_1 and X_2. For the data without multisine both models achieve similar results. Fig. 8 shows this on record I1, and Fig. 9 shows that the final model is able to capture the resonance of the system, which is not modelled in the baseline. Appendix C compares the final model on the validation records with plain training of the added states and with zero and eight added states.

B. Interpretability

- Combination errors, full model, physical part and overlaps, $\mathcal{M}_{\mathrm{U}}$ against $\mathcal{M}_{\mathrm{OBC}}$ from the detuned start (Table III)
- Correct start and the models without added states in the appendix
- Uniqueness from the two starts (?: row requested, revnote at the end)

Table III compares $\mathcal{M}_{\mathrm{OBC}}$ with $\mathcal{M}_{\mathrm{U}}$, both from the detuned start (Aspect 2). Both models have deviated further from the true combinations, than their initial values. $\mathcal{M}_{\mathrm{U}}$ deviates close

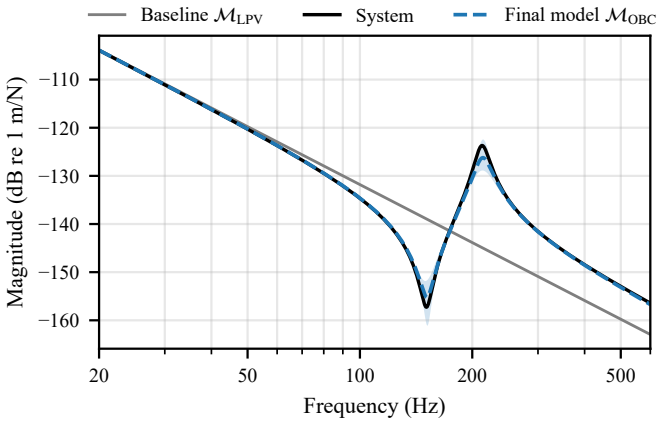


Fig. 9: Bode magnitude from the stage force F_Y to the stage position Y, linearised at rest. Geometric mean over $Y = -0.39$ to 0.39 m; shaded: range of the final model over Y.

TABLE III: Relative error of the ten combinations, unconstrained and OBC, from the detuned start.

	Start	Unconstr.	OBC[†]
Relative error (%)			
$k_{b,\Sigma}$	−10.0	−21.8	−7.0
c_{g1}	+10.0	+45.6	+26.5
c_{g2}	+10.0	+45.9	+23.3
c_y	+10.0	+43.3	+30.2
$c_{b,\Sigma}$	−10.0	+14.7	+7.0
m_h	−10.0	−1.3	−2.5
m_Σ	+10.0	+2.6	+1.1
m_Δ	−0.5	−0.5	−1.4
J_{eff}	−10.0	−2.2	−3.4
d	−10.0	−11.7	+1.0
Norm	30.0	83.1	47.7
NRMS_e (%), test records			
Full model, X_1		5.7	6.4
Full model, X_2		5.6	6.4
Full model, Y		4.8	4.6
Physical part, X_1		10.3	9.7
Physical part, X_2		9.3	10.2
Physical part, Y		41.8	42.7
Network overlap $\rho(F_{\text{aug}})$		0.84	0.13
Truth overlap $\rho(\Xi)$		0.59	
Learned pole (Hz)		212	212
Learned damping ζ		0.055	0.058

[†] R1: preliminary, update 19500 of 33400

to double that of $\mathcal{M}_{\text{OBC}}$. The combinations describing the damping deviate the most, while the mass combinations stay close to their true values. The final model with the orthogonality constraint and the unconstrained model perform equally well. OBC keeps the network out of the parameter directions: its network overlap is much lower than that of $\mathcal{M}_{\text{U}}$. The missing dynamics are not orthogonal to these directions, their overlap is 0.59. From the correct start, OBC also ends closer to the true values (Appendix C). Without added states, $\mathcal{M}_{\text{U}}$ ends closer instead.

C. Generalisation

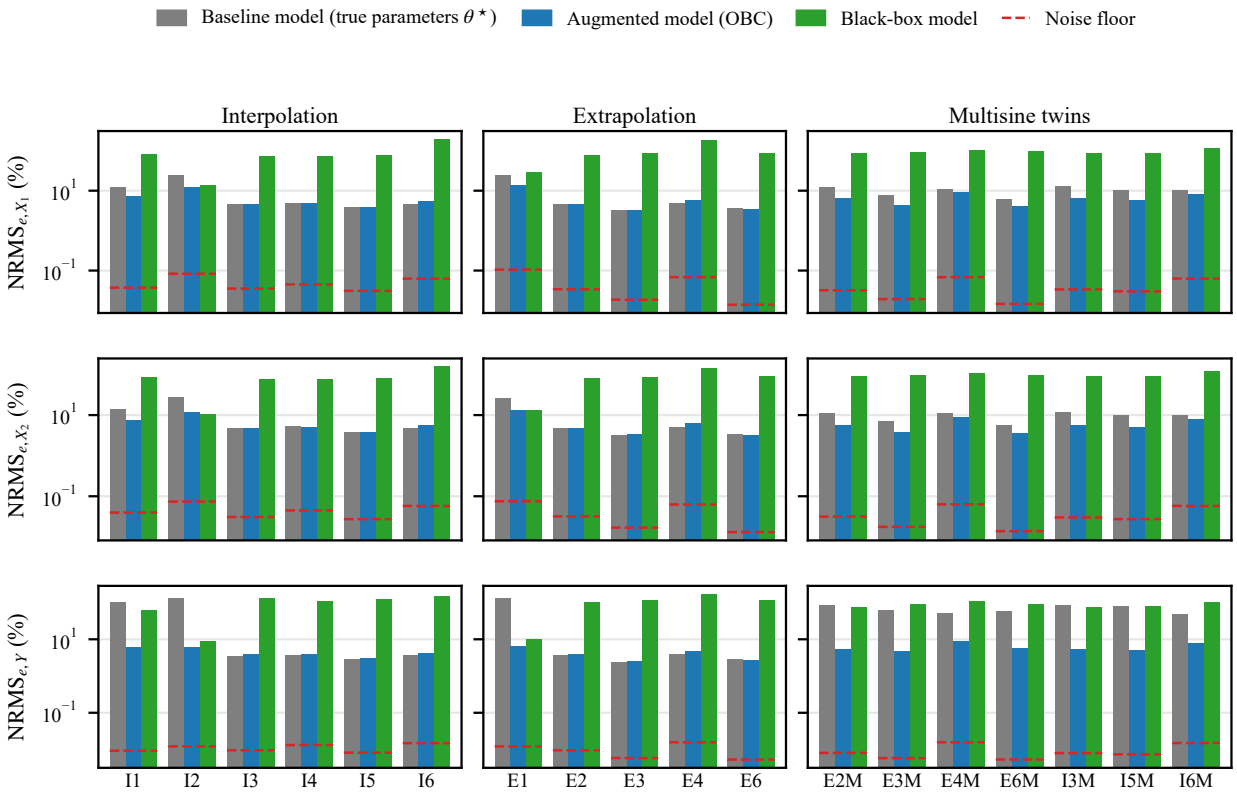
- Final model, baseline and black box per test record, each E record beside its I record (Fig. 10)
- Multisine twins of the records without multisine

The error per test record are shown in Fig. 10, for extrapolation (E) and interpolation (I) (Aspect 3). Without multisine the model performs as well or slightly above the baseline. The black box has 74 to 193 % on the records without multisine, and it is below the final model only on I2 X_2. On the multisine twins, the final model stays below 10 % on Y, while the baseline and the black box are above 47 %. [**Note:** Should I drop Fig. 10 and put it in a table instead, also due to the noise floor i think it undersells the results compared to Fig. 8.]

D. Discussion

from VIII, M-p15 The improvement on the multisine records comes mainly from the learned absorber mode (Fig. 9). The results in Table VI (Appendix C) show that both the added states and their PS initialisation are needed to obtain this improvement. Without multisine, the final model performs slightly worse than the baseline. The baseline is already accurate on these records, so the learned correction might overfit on the multisine records and introduce small errors on the records without multisine.
OBC reduces the final combination error compared with the unconstrained model (47.7 against 83.1 %), but the parameters still end up further from the truth than at their initial values (Table III). The missing dynamics have a truth overlap of 0.59 with the parameter directions. The truth overlap of 0.59 means $\Phi^\top \Xi \neq 0$, so the recovery condition of Section V-C is not met and OBC is not expected to recover the true values. [**TODO:** Use this?] The orthogonality holds only to first order and at $\bar{x} = 0$. During training, the added states enter the physical states through f_{aug} without projection. This is a limitation of imposing the constraint at $\bar{x} = 0$. With OBC the estimates from the true and detuned starts are closer together, but still differ (Table VII). The results therefore show less dependence on initialisation, but does not establish uniqueness.
The black box model performs poorly on its training records and had not converged within the available budget (Table IX). The black box model is also randomly initialised and not trained with the known controller. Having a disadvantages starting position. t
The results are limited to simulation with one seed and the same controller used to generate the data. Training and evaluation are both performed in closed loop, where the controller can suppress errors in the plant model. Through the feedback, the recorded input is correlated with the measurement noise, so the consistency of the estimate is not shown. It remains unclear whether the improvements would hold with a different controller or on measured data. end

[**TODO:** Missing: the numbers of $\mathcal{M}_{\text{OBC}}$ (R1) in this section are preliminary (update 19500 of 33400). Candidate: update them after the final R1/R2 rerun (basis: handoff 2026-10-18, Sec. 3).] [**TODO:** Missing: the network overlap of $\mathcal{M}_{\text{OBC}}$ in Table III comes from update 18200 while labelled 19500, so



augmented model (R1): preliminary, update 19500 of 33400

Fig. 10: Simulation error RMS per test record, mean over X_1, X_2, Y.

the text gives no number for it. Candidate: recheck after the final R1 rerun (basis: handoff 2026-10-18, Sec. 12).] [**TODO:** Missing: the caption of Fig. 10 says "RMS, mean over X_1, X_2, Y", while the text reads NRMS_e per axis. Candidate: check the figure and update the caption (Jasper p9: say error of what with respect to what).] [**TODO:** Are state Gramians useful for explaining each added state's contribution, or would intervention-based ablation be more direct and less linearisation-dependent?]

VIII. Conclusion

- Discussion per aspect: explanation, limit and test of each result; then the limits and departures that bound all three (handoff 2026-10-18)
- Conclusion: the answer per aspect and to the research question, the central limit, one recommendation

[Changes: black = original, blue = original, moved here (from … end), green = changed or new, ~~struck~~ = deleted ("moved to" = moved, not deleted), superscript = comment id.]

~~The improvement on the multisine records comes mainly from the learned absorber mode (Fig. 9). The results in Table VI (Appendix C) show that both the added states and their PS initialisation are needed to obtain this improvement. Without multisine, the final model performs slightly worse than the baseline. The baseline is already accurate on these records, so the learned correction might overfit on the multisine records and introduce small errors on the records without multisine.~~[moved to VII-D, M-p15] ~~OBC reduces the final combination error compared with the unconstrained model (47.7 against 83.1 %), but the parameters still end up further from the truth than at their initial values (Table III). The missing dynamics have a truth overlap of 0.59 with the parameter directions. The truth overlap of 0.59 means $\Phi^\top \Xi \neq 0$, so the recovery condition of Section V-C is not met and OBC is not expected to recover the true values. The orthogonality holds only to first order and at $\bar{x} = 0$. During training, the added states enter the physical states through f_{aug} without projection. This is a limitation of imposing the constraint at $\bar{x} = 0$. With OBC the estimates from the true and detuned starts are closer together, but still differ (Table VII). The results therefore show less dependence on initialisation, but does not establish uniqueness.~~[moved to VII-D, M-p15] ~~The black box model performs poorly on its training records and had not converged within the available budget (Table IX). The black box model is also randomly initialised and not trained with the known controller. Having a disadvantages starting position. t~~[moved to VII-D, M-p15] ~~The results are limited to simulation with one~~

~~seed and the same controller used to generate the data. Training and evaluation are both performed in closed loop, where the controller can suppress errors in the plant model. It remains unclear whether the improvements would hold with a different controller or on measured data.~~ moved to VII-D, M-p15

A parallel augmentation with added states, initialised by PS and trained in closed loop, learns the missing dynamics and lowers the Y error on the records with multisine from 116.6 to 6.2 % (Aspect 1). Without multisine it stays at the level of the baseline. OBC makes the estimates less sensitive to the start than unconstrained estimation, without loss of accuracy, but does not establish a unique estimate (Aspect 2). The augmented model keeps its accuracy outside the training range and is more accurate than the black box on every test record except X_2 of I2 (Aspect 3). The physics-based plant model should therefore be augmented in parallel with added states, initialised by PS, and with OBC. This improves the prediction where the missing dynamics are excited and partly preserves interpretability. These results are from simulation with one seed. The next step is an evaluation under a changed controller on measured data. [**TODO:** Missing: the R1 and R2 numbers are preliminary (update 19500 of 33400). Candidate: update 47.7 and 6.2 after the final rerun.]

REFERENCES

[1] I. García-Herreros, X. Kestelyn, J. Gomand, R. Coleman, and P.-J. Barre, "Model-based decoupling control method for dual-drive gantry stages: A case study with experimental validations," *Control Engineering Practice*, vol. 21, no. 3, pp. 298–307, 2013.

[2] C. S. Teo, K. K. Tan, S. Y. Lim, S. Huang, and E. B. Tay, "Dynamic modeling and adaptive control of a h-type gantry stage," *Mechatronics*, vol. 17, no. 7, pp. 361–367, 2007.

[3] J. Willard, X. Jia, S. Xu, M. Steinbach, and V. Kumar, "Integrating scientific knowledge with machine learning for engineering and environmental systems," *ACM Computing Surveys*, vol. 55, no. 4, pp. 1–37, Nov. 2022.

[4] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Learning-based augmentation of first-principle models: A linear fractional representation-based approach," 2026, submitted to Automatica.

[5] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," *Nature Reviews Physics*, vol. 3, no. 6, pp. 422–440, Jun. 2021.

[6] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics informed deep learning (part i): Data-driven solutions of nonlinear partial differential equations," *arXiv preprint arXiv:1711.10561*, 2017.

[7] B. M. Györök, M. Schoukens, T. Péni, and R. Tóth, "Orthogonal-by-construction augmentation of physics-based input-output models," *IFAC Journal of Systems and Control*, vol. 35, p. 100376, 2026.

[8] J. P. Noël and J. Schoukens, "Grey-box state-space identification of nonlinear mechanical vibrations," *International Journal of Control*, vol. 91, no. 5, pp. 1118–1139, 2018.

[9] B. M. Kessels, "Ai-based model updating for nonlinear dynamical systems," PhD thesis, Eindhoven University of Technology, Eindhoven, The Netherlands, 2025.

[10] A. Retzler, R. Tóth, M. Schoukens, G. I. Beintema, J. Weigand, J.-P. Noël, Z. Kollár, and J. Swevers, "Learning-based augmentation of physics-based models: an industrial robot use case," *Data-Centric Engineering*, vol. 5, p. e12, 2024.

[11] R. Tóth, *Modeling and Identification of Linear Parameter-Varying Systems*, ser. Lecture Notes in Control and Information Sciences. Berlin, Heidelberg: Springer, 2010, vol. 403.

[12] R. Drenth, J. H. Hoekstra, M. Schoukens, and R. Tóth, "Efficient learning of affine and rational dependency LPV models with linear fractional representation," 2026.

[13] A. Ovchinnikov, A. Pillay, G. Pogudin, and T. Scanlon, "Computing all identifiable functions of parameters for ODE models," *Systems & Control Letters*, vol. 157, p. 105030, 2021.

[14] M. Gautier and W. Khalil, "Direct calculation of minimum set of inertial parameters of serial robots," *IEEE Transactions on Robotics and Automation*, vol. 6, no. 3, pp. 368–373, 1990.

[15] R. Drenth, "Efficient gradient-based learning of LPV models with linear fractional representation," Master's thesis, Eindhoven University of Technology, 2025.

[16] G. I. Beintema, M. Schoukens, and R. Tóth, "Deep subspace encoders for nonlinear system identification," *Automatica*, vol. 156, p. 111210, 2023.

[17] U. Forssell and L. Ljung, "Closed-loop identification revisited: Updated version," Department of Electrical Engineering, Linköping University, Tech. Rep. LiTH-ISY-R-2021, Apr. 1998.

[18] J. H. Hoekstra, B. M. Györök, R. Tóth, and M. Schoukens, "Encoder initialisation methods in the model augmentation setting," 2026.

[19] A. Hefny, C. Downey, and G. J. Gordon, "Supervised learning for dynamical system learning," in *Advances in Neural Information Processing Systems*, vol. 28, 2015.

[20] C. Downey, A. Hefny, B. Li, B. Boots, and G. J. Gordon, "Predictive state recurrent neural networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[21] A. Venkatraman, N. Rhinehart, W. Sun, L. Pinto, M. Hebert, B. Boots, K. M. Kitani, and J. A. Bagnell, "Predictive-state decoders: Encoding the future into recurrent networks," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[22] B. M. Györök, J. H. Hoekstra, J. Kon, T. Péni, M. Schoukens, and R. Tóth, "Orthogonal projection-based regularization for efficient model augmentation," in *Proceedings of the 7th Annual Conference on Learning for Dynamics and Control*, ser. Proceedings of Machine Learning Research, N. Ozay, L. Balzano, D. Panagou, and A. Abate, Eds., vol. 283, 2025, pp. 166–178.

TABLE IV: Nominal baseline parameters and, in the lower block, the parameters of the simulated system only.

Parameter	Value	Parameter	Value
m_1	10.2 kg	m_2	10.7 kg
m_b	22.8 kg	m_h	10.1 kg
J_b	1.00 kg m^2	J_h	0.05 kg m^2
c_{g1}	14.5 N s/m	c_{g2}	20.3 N s/m
c_y	10.0 N s/m	d	0.10 m
c_{b1}	9.0 N m s/rad	c_{b2}	9.0 N m s/rad
k_{b1}	1987.5 N m/rad	k_{b2}	1987.5 N m/rad
L_b	0.725 m		
c_{c,X_1}	16.80 N	c_{c,X_2}	18.35 N
$c_{c,Y}$	11.60 N	v_{fr}	1 mm/s
m_a	0.5 m_h	L_0	0.10 m
f_a	150 Hz	ζ_a	0.03

APPENDIX A
SUPPORTING DERIVATIONS

A. Nominal physical parameters

Table IV lists the nominal values used by the baseline. They are taken from the gantry reported by García-Herreros et al. [1, Table I]. The lower block holds the parameters of the simulated system that the baseline lacks (Section VI-A).

B. Explicit LPV-LFR realisation

In (7), M_0 is (5) evaluated at $Y = 0$, and

$$M_1 = \begin{bmatrix} 0 & -m_h & 0 \\ -m_h & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \qquad M_2 = \begin{bmatrix} 0 & 0 & 0 \\ 0 & m_h & 0 \\ 0 & 0 & 0 \end{bmatrix}.$$

With a_1 and a_2 from (9) treated as independent signals, solving $M_0\ddot{q}+M_1a_1+M_2a_2 = u-C\dot{q}-Kq$ for $\ddot{q}$ with $R = M_0^{-1}$ gives $\ddot{q} = R(u - Kq - C\dot{q} - M_1a_1 - M_2a_2)$. With $\dot{\tilde{x}} = [\dot{q}^\top \ \ddot{q}^\top]^\top$, $z = [\ddot{q}^\top \ a_1^\top]^\top$, and $w = [a_1^\top \ a_2^\top]^\top$, the LTI part G of (12), using a continuous-time adaptation of the block partition in [12, Eq. (2)], is

$$\begin{bmatrix} \dot{\tilde{x}} \\ z \\ q \end{bmatrix} = \underbrace{\begin{bmatrix} A_x & B_w & B_u \\ C_z & D_{zw} & D_{zu} \\ C_y & D_{yw} & D_{yu} \end{bmatrix}}_{G} \begin{bmatrix} \tilde{x} \\ w \\ u \end{bmatrix}. \tag{46}$$

with blocks

$$\begin{aligned} A_x &= \begin{bmatrix} 0 & I_3 \\ -RK & -RC \end{bmatrix}, & B_w &= \begin{bmatrix} 0 & 0 \\ -RM_1 & -RM_2 \end{bmatrix}, & B_u &= \begin{bmatrix} 0 \\ R \end{bmatrix}, \\ C_z &= \begin{bmatrix} -RK & -RC \\ 0 & 0 \end{bmatrix}, & D_{zw} &= \begin{bmatrix} -RM_1 & -RM_2 \\ I_3 & 0 \end{bmatrix}, & D_{zu} &= \begin{bmatrix} R \\ 0 \end{bmatrix}, \\ C_y &= [I_3 \ \ 0], & D_{yw} &= 0, & D_{yu} &= 0. \end{aligned} \tag{47}$$

Here $\tilde{x}, z, w \in \mathbb{R}^6$ and $u, q \in \mathbb{R}^3$, so $G \in \mathbb{R}^{15\times 15}$. The actuator frame enters only through $u = Pu_{\text{act}}$ and $q_{\text{act}} = P^\top q$ in (3).

Substituting $w = \Delta(Y)z$ into the z equation of (46) gives the algebraic loop equation $(I_6 - D_{zw}\Delta(Y))z = C_zx + D_{zu}u$. Written out,

$$\begin{bmatrix} I_3 + YRM_1 & YRM_2 \\ -YI_3 & I_3 \end{bmatrix} \begin{bmatrix} \ddot{q} \\ a_1 \end{bmatrix} = \begin{bmatrix} R(u - Kq - C\dot{q}) \\ 0 \end{bmatrix}.$$

The second block row gives $a_1 = Y\ddot{q}$, and the first then gives $M(Y)\ddot{q} = u - C\dot{q} - Kq$, that is (4), after multiplication by M_0. The condition of [12, Def. 1], that $I_6 - D_{zw}\Delta(Y)$ is nonsingular, is therefore the condition that $M(Y)$ is nonsingular: the Schur complement of the lower identity block gives

$$\begin{aligned} \det\big(I_6 - D_{zw}\Delta(Y)\big) &= \det\big(I_3 + YRM_1 + Y^2RM_2\big) \\ &= \det M(Y)/\det M_0. \end{aligned}$$

Since $M(Y)$ is nonsingular, the loop is solved in closed form, $\ddot{q} = M(Y)^{-1}(u - C\dot{q} - Kq)$ with

$$M(Y)^{-1} = \frac{N_0 + YN_1 + Y^2N_2}{d(Y)}, \qquad d(Y) = \det M(Y),$$

where $N_0 + YN_1 + Y^2N_2 = \operatorname{adj} M(Y)$ and $d(Y)$ are both quadratic in Y. The implementation evaluates the numerator with Horner's scheme and $d(Y)$ term by term. **[TODO:** Missing: print N_0, N_1, N_2 and $d(Y)$ explicitly? Candidate: no, appendix budget; the quadratic degree was checked symbolically (sympy, 2026-10-06).]

C. Positive definiteness of the inertia matrix

With $a = L_bm_\Delta/2$, elimination of the m_h block and completion of the square give

$$\begin{aligned} \frac{\det M(Y)}{m_h} &= (m_\Sigma + m_h)(J_{\text{eff}} + m_hY^2) - (a - m_hY)^2 \\ &= m_\Sigma m_h \left(Y + \frac{a}{m_\Sigma}\right)^2 \\ &\quad + (m_\Sigma + m_h)\left(J_{\text{eff}} - \frac{a^2}{m_\Sigma}\right). \end{aligned}$$

The 2×2 Schur complement has positive leading entry $m_\Sigma + m_h$. Sylvester's criterion therefore reduces positive definiteness to positivity of the completed-square expression for every Y. Hence, for positive m_h, m_Σ, and J_{eff}, $M(Y) \succ 0$ for all $Y \in \mathbb{R}$ if and only if (14) holds. For the physical parametrisation, the difference between the two sides is

$$\begin{aligned} m_\Sigma J_{\text{eff}} - \frac{L_b^2}{4}m_\Delta^2 &= m_\Sigma(J_b + J_h) \\ &\quad + \frac{L_b^2}{4}\left[4m_1m_2 + m_b(m_1 + m_2)\right], \end{aligned}$$

which is strictly positive for positive component masses and yaw inertias.

In (15), let $b(\theta_{\text{base}}) = \gamma(2/L_b)\sqrt{m_\Sigma J_{\text{eff}}}$. The offset and scale are

$$r_0 = \frac{m_{\Delta,0}}{b(\theta^0_{\text{base}})}, \quad \eta_0 = \operatorname{atanh}(r_0), \quad s = \frac{m_{\Delta,0}}{b(\theta^0_{\text{base}})(1 - r_0^2)}.$$

Here η_0 makes $\beta_\Delta = 0$ reproduce the initial combination, and s gives $\partial m_\Delta/\partial\beta_\Delta|_0 = m_{\Delta,0}$, matching the local relative-change scaling of the logarithmic coordinates. The construction requires $m_{\Delta,0} \neq 0$; a start near zero makes s small and the coordinate numerically inert. The exact inverse used for reporting is

$$\beta_\Delta = \frac{\operatorname{atanh}(m_\Delta/b(\theta_{\text{base}})) - \eta_0}{s}.$$

D. Identifiable combinations

For $m_h > 0$, the ten distinct entries of M_0, C and K determine θ_{base} of (13) uniquely (Section III-C):

$$m_h = M_{0,33}, \qquad d = -M_{0,23}/m_h, \qquad m_\Sigma = M_{0,11} - m_h,$$
$$m_\Delta = 2M_{0,12}/L_b, \qquad J_{\text{eff}} = M_{0,22} - m_h d^2, \qquad k_{b,\Sigma} = K_{22},$$
$$c_{g1} = \tfrac{1}{2}C_{11} + C_{12}/L_b, \qquad c_{g2} = \tfrac{1}{2}C_{11} - C_{12}/L_b, \qquad c_y = C_{33},$$
$$c_{b,\Sigma} = C_{22} - \tfrac{L_b^2}{4}C_{11}.$$

Conversely, (5) and (6) give every entry as a function of θ_{base}, so the map is one-to-one.

[**TODO:** Remarks cut from Section II-B; keep here or drop?]

- Y^2 is realised by repeating Y, not as a second scheduling variable $p_2 = Y^2$, which would admit pairs with $p_2 \neq Y^2$; extra scheduling variables cause overbounding in controller synthesis [12]
- The scheduling map is a known selection of the state ($Y = q_3$), not a learned map as in the self-scheduled models of [12]
- The exact condition $\det M(Y) \neq 0$ replaces the sufficient condition of [12, Thm. 6], which needs a scheduling set in the unit ball and $\rho(D_{zw}) < 1$
- Drenth's Def. 1 is stated in discrete time but constrains only the algebraic loop equation, so it applies unchanged in continuous time

E. PS stopping rule

The unexplained fraction ν of (30) is evaluated after every i_cth update of V, with ν_i its value after update i. S1 ends at update

$$i_1 = \min\big\{ i \in \mathcal{J} :\ i = i_{\max} \text{ or } \big(i \geq i_{\min} \text{ and } \nu_i^{\text{new}} > (1-\delta_\nu)\,\nu_i^{\text{old}}\big)\big\},$$
$$\nu_i^{\text{new}} = \min_{j \in \mathcal{W}_i} \nu_j, \qquad \nu_i^{\text{old}} = \min_{j \in \mathcal{J}_i \setminus \mathcal{W}_i} \nu_j,$$

where $\mathcal{J} = \{i_c, 2i_c, \dots\}$, $\mathcal{J}_i = \{j \in \mathcal{J} : j \leq i\}$, $\mathcal{W}_i$ the last w elements of $\mathcal{J}_i$, δ_ν the plateau tolerance and $i_{\min}$, $i_{\max}$ the earliest and latest end of S1, with $\nu_i^{\text{old}} = \infty$ while $\mathcal{J}_i$ has at most w elements. S2 runs if $\nu_{i_1} \leq \nu_{\max}$ and, when $i_1 > i_s$, also $\nu_{i_s} \leq \nu_{\max}$, where $i_s < i_{\max}$ is a fixed early check. If $\nu_{i_s} > \nu_{\max}$, PS ends at i_s. The values are given in Table XII.

F. Open-loop frequency response

- Full 3×3 response of Fig. 9: same linearisation at rest, same seven positions (`Code/Evaluation/eval_frf.py`)
- On the X entries the final model, the system and the baseline coincide; the spread over Y is the position dependence of the system
- The final model has a small resonance near 212 Hz in the cross terms between X and Y that the system lacks (state it)

Fig. 11 gives all nine entries of the response in Fig. 9, with the same linearisation and positions.

TABLE V: NRMS_e per axis on the training, validation and test records, mean over each set, with and without multisine. Physical part: the trained model with the learned block switched off.

	Multisine			No multisine		
NRMS_e (%)	X_1	X_2	Y	X_1	X_2	Y
Training records						
Baseline, true θ	21.9	24.5	120.0	4.6	5.0	3.7
Augmented, unconstrained	9.4	10.2	6.6	4.1	4.3	4.3
physical part	21.0	23.7	120.8	5.9	4.3	4.2
Augmented, OBC (final)†	10.5	11.2	6.8	4.5	5.0	4.2
physical part†	23.9	26.9	122.3	4.7	4.6	5.1
Black-box model	32.4	34.5	25.2	75.6	76.9	104.4
Noise floor	0.06	0.07	0.01	0.04	0.04	0.01
Validation records						
Baseline, true θ	23.0	24.9	121.8	4.7	4.9	3.7
Augmented, unconstrained	9.9	10.2	6.6	4.1	4.2	4.3
physical part	22.1	24.1	122.7	5.9	4.2	4.2
Augmented, OBC (final)†	11.0	11.1	7.1	4.6	4.9	4.1
physical part†	25.3	27.4	123.9	4.8	4.5	5.0
Black-box model	31.6	33.6	23.8	75.6	76.9	103.3
Noise floor	0.06	0.07	0.01	0.04	0.04	0.01
Test records I1 to I6						
Baseline, true θ	18.8	20.7	116.6	4.5	4.6	3.4
Augmented, unconstrained	8.8	8.4	6.3	4.2	4.2	4.0
physical part	17.9	19.4	117.4	6.5	4.2	3.9
Augmented, OBC (final)†	9.7	9.5	6.2	4.7	4.8	3.8
physical part†	19.8	22.0	118.7	4.7	4.3	4.7
Black-box model	47.8	47.8	35.9	104.8	97.0	127.7
Noise floor	0.06	0.06	0.01	0.04	0.04	0.01

† preliminary, update 19500 of 33400

Appendix B
Training, validation and test results

Table V gives the models of Section VII on the training, validation and test records.

Appendix C
Design choices

This appendix gives the results behind the choices of the baseline form, the augmentation structure, the start of the added states and the black box. Fig. 12 compares the LPV baseline with the LTI baseline frozen at $Y = 0$. [**Note:** Add the validation curves here: validation NRMS_e against the optimiser updates for every grey-box run and the selected black box]

Appendix D
Experiment design details

This appendix gives the details of Section VI and the checks of its design choices. The checks use the known simulated system where the text says so.

A. Data-generating system

- The controller K_1 of every record
- The noise spectrum, fitted to the standstill servo error of the machine (?: confidentiality)
- The definition of the signal-to-noise ratio

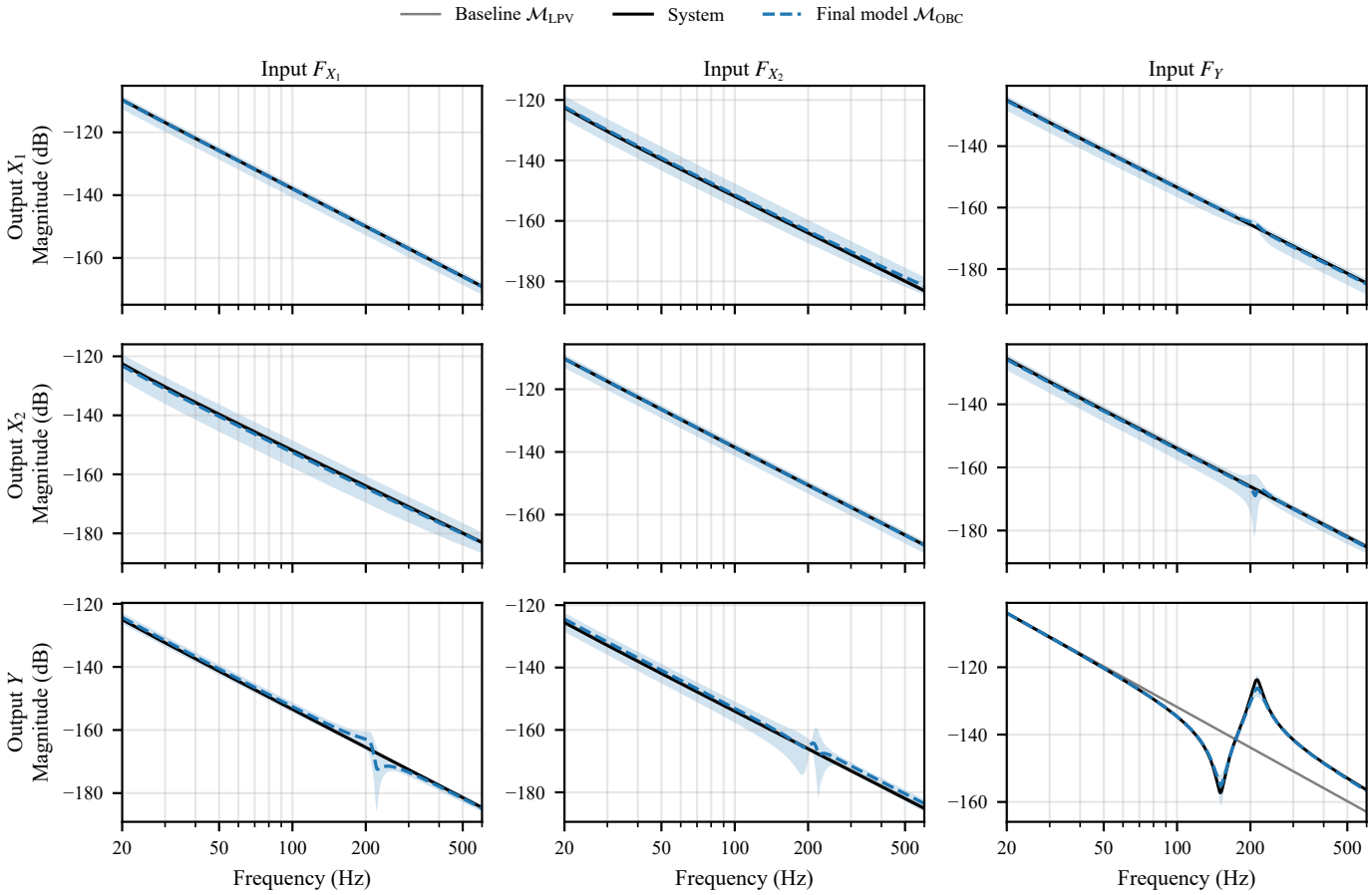


Fig. 11: Bode magnitude from the stage forces (columns) to the stage positions (rows), linearised at rest. Geometric mean over $Y = -0.39$ to 0.39 m; shaded: range of the final model over Y.

The controller of every record is, per stage axis j,

$$K_{1,j}(s) = \kappa_j \, \frac{s+\omega_b/6}{s} \, \frac{s+\omega_b/3}{s+3\omega_b} \, \frac{10\,\omega_b}{s+10\,\omega_b}, \qquad (48)$$

where $\omega_b = 2\pi f_b$ is the bandwidth and κ_j sets the open-loop gain of axis j to one at ω_b, for the diagonal entry of the rigid baseline at $Y = 0$ from stage force to stage position. K_1 is discretised by the Tustin rule at f_s.

The noise v_j has the one-sided spectrum

$$S_{v,j}(f) = \lambda_j^2 \, |H_j(f)|^2 + \sigma_{v,j}^2, \qquad (49)$$

where H_j is a second-order Butterworth low-pass with cutoff $f_{v,j}$, λ_j its level and $\sigma_{v,j}$ a white floor. λ_j and $f_{v,j}$ are fitted so that the simulated standstill servo error at $Y = 0$ matches the measured one of the industrial machine, and $\sigma_{v,j}$ is its level above 5 kHz. The signal-to-noise ratio of Section VI-A is $\mathrm{SNR}_j = 20\log_{10}\big(\mathrm{RMS}(y_j^\circ - r_j)/\mathrm{RMS}(y_j - y_j^\circ)\big)$ over the training records.

B. Excitation and records

- The multisine: grid from the width of the anti-resonance, best crest factor of 30 draws, amplitudes, no record needed scaling
- The upper band edge and the band over the frequency response of the simulated system
- The five types of motion profile, the operational motion profiles of the test records and the operating range of all records

In each channel $c \in \{X, \Theta, Y\}$ of the generalised force, the multisine is

$$u_c^{\mathrm{ms}}(t) = A_c \sqrt{\frac{2}{|\mathcal{L}|}} \sum_{l \in \mathcal{L}} \cos(2\pi l\,\Delta f\, t + \alpha_{c,l}), \qquad (50)$$

where $\Delta f = 0.005\, f_b$, $\mathcal{L} = \{212, \ldots, 594\}$ spans $1.06\, f_b$ to $2.97\, f_b$, and the phases $\alpha_{c,l}$ are drawn uniformly per record. It enters at the actuators as $u^{\mathrm{ff}} = P^{-1} u^{\mathrm{ms}}$. The grid places 18 lines inside the half-power width $2\zeta_a f_a = 0.09\, f_b$ of the anti-resonance, the narrowest feature of the system, and makes the multisine periodic in 2 s. Of 30 phase draws, the one with the lowest crest factor is kept; for Θ the crest factor is taken on the simulated yaw response. The rms amplitudes A_c keep every record within the force and stroke limits of the machine without scaling. K_1 is tuned for f_b on the rigid baseline at $Y = 0$. On the simulated system, the crossover of the loop lies between $0.75\, f_b$ and $1.06\, f_b$ over the axes and the training positions, so the band starts at the highest. The

TABLE VI: NRMS_e per axis of every model, mean over the validation records, with and without multisine.

	Multisine			No multisine		
NRMS_e (%)	X_1	X_2	Y	X_1	X_2	Y
Baseline, true θ	23.0	24.9	121.8	4.7	4.9	3.7
Baseline, detuned $\theta^\dagger$	30.6	34.0	131.2	10.2	5.5	11.4
Static parallel, n_a=0	14.3	14.3	86.5	4.9	4.6	18.4
Static parallel, n_a=0, detuned θ	14.2	14.2	86.5	5.0	4.7	18.7
Static parallel (OBC), n_a=0, detuned θ	14.9	14.9	86.6	5.5	5.2	19.7
Plain training, n_a=2	11.6	12.0	78.1	4.5	4.6	14.6
PS init., n_a=2	9.9	10.5	6.4	4.1	4.3	4.3
PS init., baseline part only	21.1	21.6	120.5	5.5	4.3	3.7
PS init., n_a=8	9.5	9.9	6.3	4.1	4.2	4.3
PS init., detuned θ	9.9	10.2	6.6	4.1	4.2	4.3
Augmented (OBC), detuned $\theta^\dagger$	11.0	11.1	7.1	4.6	4.9	4.1
Black-box model	31.6	33.6	23.8	75.6	76.9	103.3
Noise floor	0.06	0.07	0.01	0.04	0.04	0.01

† preliminary, update 19500 of 33400

TABLE VII: Relative error of the ten combinations, unconstrained and OBC, from the detuned and the correct start.

	Detuned start			Correct start	
	Start	Unconstr.	OBC†	Unconstr.	OBC†
Relative error (%)					
$k_{b,\Sigma}$	−10.0	−21.8	−7.0	−7.6	−5.6
c_{g1}	+10.0	+45.6	+26.5	+27.7	+15.8
c_{g2}	+10.0	+45.9	+23.3	+27.2	+13.5
c_y	+10.0	+43.3	+30.2	+29.5	+18.9
$c_{b,\Sigma}$	−10.0	+14.7	+7.0	+29.2	+19.3
m_h	−10.0	−1.3	−2.5	+1.4	+2.6
m_Σ	+10.0	+2.6	+1.1	+1.3	−0.5
m_Δ	−0.5	−0.5	−1.4	−0.1	−1.0
J_{eff}	−10.0	−2.2	−3.4	+4.1	+4.7
d	−10.0	−11.7	+1.0	−9.1	−5.7
Norm	30.0	83.1	47.7	58.3	35.4
NRMS_e (%), test records					
Full model, X_1		5.7	6.4	5.8	6.4
Full model, X_2		5.6	6.4	5.8	6.5
Full model, Y		4.8	4.6	4.7	4.6
Physical part, X_1		10.3	9.7	10.0	9.2
Physical part, X_2		9.3	10.2	9.0	9.4
Physical part, Y		41.8	42.7	40.7	40.7
Network overlap $\rho(F_{\mathrm{aug}})$		0.84	0.13	0.85	0.10
Truth overlap $\rho(\Xi)$			0.59		
Learned pole (Hz)		212	212	212	212
Learned damping ζ		0.055	0.058	0.056	0.058

† R1, R2: preliminary, update 19500 of 33400

upper edge follows from the best linear approximation of the simulated system with friction under K_1, measured at the five training positions. Above the crossover, the band holds 90 % of the squared difference between this response and that of the baseline, counted where the difference exceeds its uncertainty. A rerun with the friction law of (43a) gives the same edges. [**TODO:** Missing: reason for the 90 % share of the upper edge and for 30 phase draws, and a cite key for the best linear approximation. Candidate: keep both as heuristic (D-217, D-219); cite Pintelon and Schoukens 2012 for the best linear approximation (no cite key yet).]

The five types of motion profile are standstill at a fixed Y, slow sweeps of Y, X-Y Lissajous paths, point-to-point moves with S-curve profiles, and operational motion profiles, which are sine-shaped moves representative of machine operation. The test records I3 to I6 add new operational motion profiles: sine-shaped routes with X and Y together, alternating X-only and Y-only moves, an S-curve route and long strokes. Table X lists the records, and Fig. 15 shows their operating range.

C. Processing and checks

- The reduction to the model rate: block mean of the force, zero-phase filter on the servo error, trim at the record ends
- The checks of rate, filter, window, controller rate and friction regime
- The cause of the controller-rate change (?)

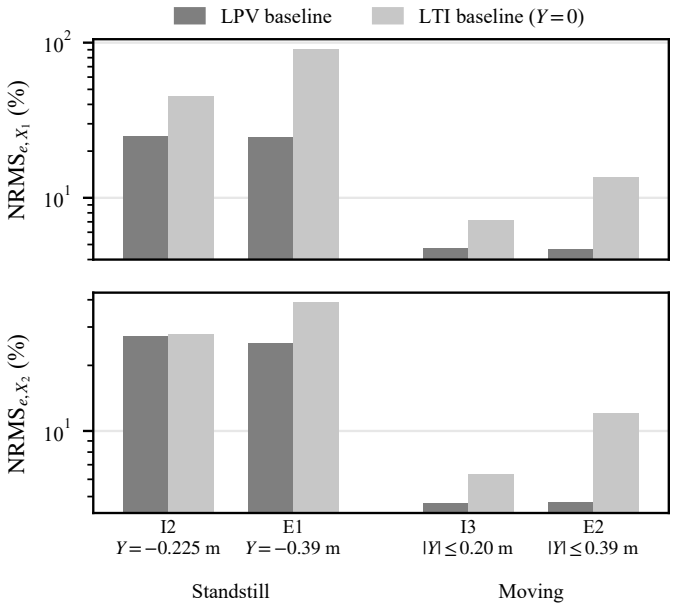


Fig. 12: Simulation error RMS on the X axes of the LPV baseline and the LTI baseline at $Y = 0$, both at the true parameters.

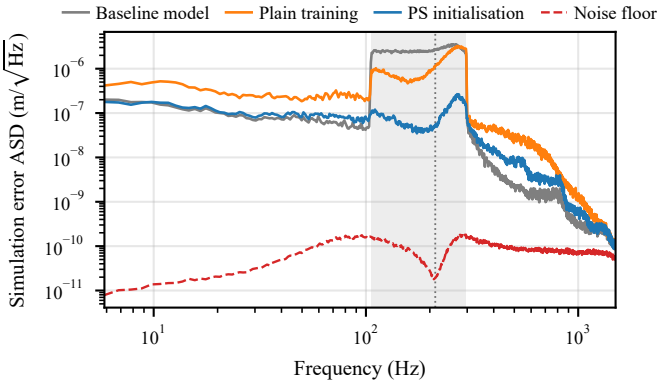


Fig. 13: Amplitude spectral density of the Y simulation error on the validation records with multisine. Shaded: multisine band. Dotted: absorber pole.

With the argument counting samples at f_s, the records are reduced to T_s by

$$u_k = \frac{1}{5}\sum_{i=0}^{4} u_{\mathrm{act}}(5k+i), \qquad y_k = r(5k) + \big(\mathcal{B} * (y-r)\big)(5k), \tag{51}$$

where $\mathcal{B}$ is an eighth-order Butterworth low-pass at 1.6 kHz, applied forward and backward. [**TODO:** Changed from: where $\mathcal{B}$ is a zero-phase eighth-order Butterworth low-pass at 1.6 kHz.] [**Note:** Noise-free twins: a noise-free TRAINING run point-samples y, $y_k = y(5k)$ (config.py:33, data.py:175-176), but the evaluation loads the twins for the noise floor with the lowpass configuration (eval_blackbox.py:117, 156-158), so every reported floor uses (51) as written. No final run used noise-free training data. Add a point-sampling sentence only if a noise-free training run is reported.] The block mean is the exact input of a zero-order-hold model at T_s. The filter acts on the servo error only, so the reference passes unchanged. It changes the noise-free output by at most $2.3\cdot10^{-9}$ m RMS. Its impulse response falls below 1 % of its peak within 2.6 ms, so the 5 ms removed at both record ends hold its edge transient. [**TODO:** Missing: source for the cutoff at 0.8 times the Nyquist frequency and for the order 8. Candidate: Gautier, Janot and Vandanjon, IEEE TCST 2013 (decimation at 0.8 of the Nyquist frequency; no cite key yet); order 8 heuristic (D-232).]

For $|v_i| \geq 3v_{\mathrm{fr}}$, the friction of (43a) is within 0.5 % of the Coulomb law. Table XI gives the checks of the design, including the share of samples in this range. The training loop re-discretises K_1 at T_s, which changes the feedback force on the training records by a median of 10.6 % RMS. [**TODO:** Missing: the cause of this change and whether it affects the results. Candidate: the half-sample offset between the block-

TABLE VIII: Relative error of the ten combinations without added states ($n_a = 0$), unconstrained and OBC, from the detuned start.

	Start	Unconstr.	OBC
Relative error (%)			
$k_{b,\Sigma}$	−10.0	−10.2	−1.5
c_{g1}	+10.0	+42.4	+39.3
c_{g2}	+10.0	+12.6	+31.7
c_y	+10.0	+2.9	+38.4
$c_{b,\Sigma}$	−10.0	−12.9	+22.2
m_h	−10.0	−7.3	−3.9
m_Σ	+10.0	−0.6	+2.9
m_Δ	−0.5	+0.4	−2.0
J_{eff}	−10.0	−4.8	−4.0
d	−10.0	−8.2	+10.9
Norm	30.0	48.8	68.4
NRMS_e (%), test records			
Full model, X_1		7.6	7.7
Full model, X_2		7.2	7.7
Full model, Y		41.8	42.7
Physical part, X_1		9.3	10.8
Physical part, X_2		12.0	10.4
Physical part, Y		46.9	43.8
Network overlap $\rho(F_{\mathrm{aug}})$		0.90	0.42
Truth overlap $\rho(\Xi)$		0.59	0.59

TABLE IX: Black-box search: the 11 configurations and their best validation NRMS_e.

#	n_x	Width	LR	Param.	NRMS_e (%)	Best update	Status
0	8	8	$3\cdot10^{-3}$	7494	63.4	8000	completed
1	8	16	$3\cdot10^{-3}$	13422	59.9	28000	completed
2	8	32	$3\cdot10^{-3}$	27582	63.4	20000	completed
3	16	8	$3\cdot10^{-3}$	10270	52.5	50000	completed
4	16	16	$3\cdot10^{-3}$	16518	56.2	11000	failed
5	16	32	$3\cdot10^{-3}$	31318	54.8	75000	completed
6	32	8	$3\cdot10^{-3}$	16206	58.2	9000	completed
7	32	16	$3\cdot10^{-3}$	23094	57.5	3000	completed
8	32	32	$3\cdot10^{-3}$	39174	59.1	4000	completed
9	**16**	**16**	$1\cdot10^{-3}$	**16518**	48.2	**78000**	**completed**
10	16	16	$1\cdot10^{-2}$	16518	66.4	1000	failed

Bold: selected on validation (lowest NRMS_e; within 2 % the fewer parameters).

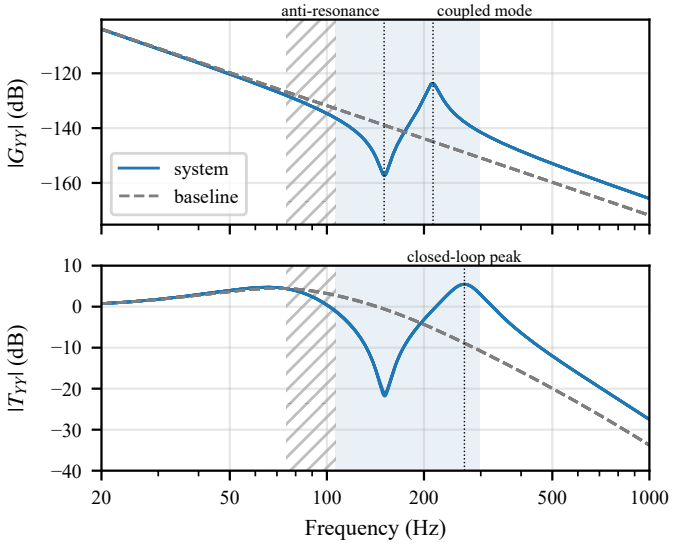


Fig. 14: Stage axis Y of the simulated system and of the baseline, linearised at the five training positions (lines overlap). Top: open loop. Bottom: complementary sensitivity with K_1. Shaded: multisine band; hatched: range of the K_1 crossovers over the axes and positions.

TABLE X: Training, validation and test records.

Record	Motion	Multisine	New	Pair
Training	all five types	14 of 18		
Validation	all five types	4 of 6	positions, draws	
I1	point-to-point	yes	phases, setpoints	
I2	standstill	yes	position	
I3	operational	no†	motion type	
I4	route	no	motion type	
I5	route	no†	motion type	
I6	route	no†	motion type	
E1	standstill	yes	$\lvert Y\rvert = 1.3\,Y_{\mathrm{tr}}$	I2
E2	operational	no†	stroke of Y	I3
E3	route	no†	acceleration, 100 %	I5
E4	route	no†	velocity, 98 %	I6
E6	operational	no†	several, 131 % acc.	I3

I: interpolation, inside the training range. E: extrapolation, outside it in the setting under New, with the motion of the Pair record. New: what differs from the training records, for E from the Pair record. E6 differs in more than one setting and is a stress record. †Also evaluated with the training multisine added (I3M, E2M, and so on), at equal amplitude within a pair and 0.30 of the training amplitude for I6 and E4.

mean force and the controller output at T_s ($2\sin(\pi f \cdot 0.1$ ms) is 9 to 19 % over the band, only records with multisine show it; not verified); the effect to Section VII or the Discussion.] [**TODO:** Cause found (2026-10-16, D-265): the check compares the 4 kHz controller output at sample kD with the force averaged over kD to $kD+4$, centred 0.1 ms later. Compared at the same instant, the two controllers agree to 0.5 to 1 % on TR-S1. The 10.8 % is the change of the force within one 0.25 ms hold, the cost of closing the loop at 4 kHz in simulation, not an error in the block-mean preprocessing (D-087), which stays. Its output effect is bounded by the oracle error of the closed-loop report.

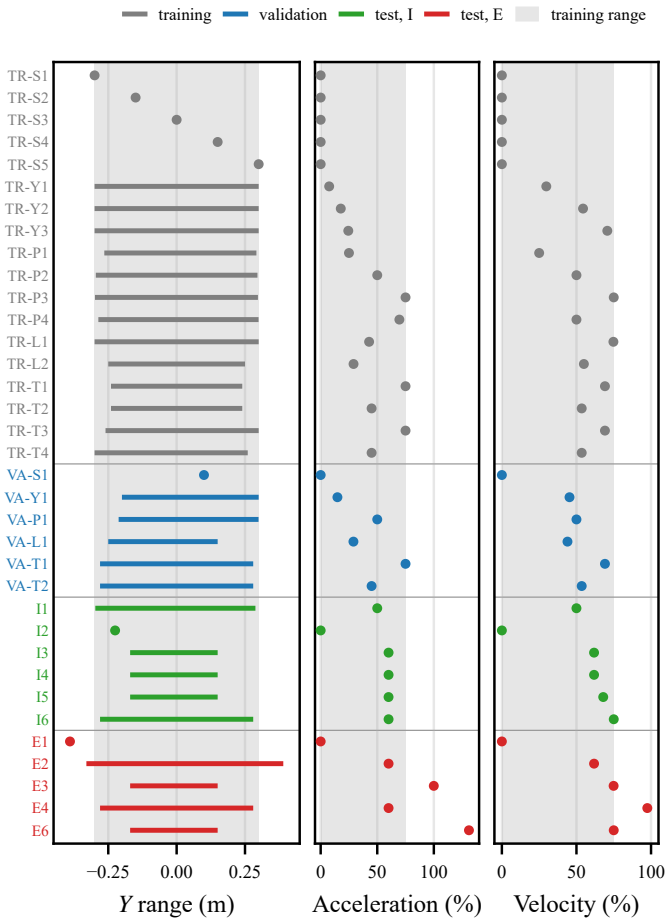


Fig. 15: Coverage of each record: range of Y (left), largest acceleration (middle) and largest velocity (right), in % of the machine limits. Shaded: range of the training records.

TABLE XI: Checks of the experiment design.

Check	Value
Simulated loop, linearised at the training positions	
Phase-margin loss, 4 / 2 kHz	3.8° / 8.6°
Slowest closed-loop time constant	11 ms
Response to a force error, 1 %	44 ms (window 100 ms)
Filter impulse response, 1 %	2.6 ms (trim 5 ms)
Training records	
Noise above 2 kHz, raw / filtered	$4.6 \cdot 10^{-9}$ / $4.0 \cdot 10^{-11}$ m
SNR 0.24 f_b to 1.05 f_b, without multisine	≥59 dB
Samples with $\lvert v\rvert \geq 3v_{\mathrm{fr}}$, multisine	≥71 %
K_1 at 20 kHz, force error	$1.5 \cdot 10^{-9}$
K_1 at T_s, force change, median	10.6 %
largest; without multisine	16.8 %; 0.6 %

The check is corrected to report both numbers separately, so these macros and this sentence change.]

D. Black-box model

- The SUBNET model of the closed loop, from reference and injected force to servo error, with its encoder

TABLE XII: Hyperparameters of the identification.

Setting	Value	Basis
Model and data		
Model rate $1/T_s$	4 kHz	derived
RK4 steps per T_s	1	heuristic
Window n_f	400 (0.1 s)	derived
Window stride	10 samples	heuristic
Encoder window $n+1$	30 samples	heuristic
Encoder start	least squares (SVD)	[18, Eq. (35)]
SVD cutoff (N windows, m regressors)	$\max(N,m)\,\epsilon$	numerical
Added states $n_{\bar{x}}$	0, 2, 8	compared
Network, encoder correction	2×16, tanh	tuned (earlier)
Precision	float64	numerical
Margin γ	0.99	heuristic
Detuning of the combinations	10 %	heuristic
Optimiser		
Adam rate, ϵ	10^{-5}, 10^{-16}	tuned (earlier)
Batch	512 windows	heuristic
Epochs	200	compute-limited
Validation interval	1300 updates	compute-limited
L-BFGS iterations, history	100, 50	inherited
L-BFGS inner steps, windows	20, 16384	compute-limited
L-BFGS tolerance (grad., change)	$2.6 \cdot 10^{-5}$, $2.6 \cdot 10^{-7}$	inherited, scaled to the loss
Seeds per model	1	compute-limited
PS (Section IV-E, Appendix A-E)		
S1 and S2 Adam rate	10^{-3}	Adam default
S1 batch, windows $\lvert\mathcal{C}\rvert$	64, 128	heuristic
Decoder width n_h, horizon H	16, $n+1$	as above
Check interval i_c	25 updates	heuristic
Plateau checks w, tolerance δ_ν	4, 0.02	heuristic
S1 updates $i_{\min}$, $i_{\max}$	100, 350	heuristic
Early check i_s, threshold $\nu_{\max}$	150, 0.90	heuristic
Held-out records	last per class with at least 3 records	heuristic
S2 pairs $N_{\mathcal{P}}$ (fit, held-out)	8192, 2048	heuristic
S2 batch, check interval	512, 100 updates	heuristic
S2 plateau (checks, tolerance)	5, 0.01	heuristic
S2 updates (min, max)	500, 8000	heuristic
Black box (Section VI-C)		
States n_x	8, 16, 32	tuned
Width of f_{bb}, h_{bb} (encoder $2\times$)	8, 16, 32	tuned
Adam rate	10^{-3}, $3 \cdot 10^{-3}$, 10^{-2}	tuned
Batch, stride	256, 25	inherited
Updates	80000	compute-limited
Selection	lowest validation NRMS_e	heuristic

The black box of Section VI-C is

$$x^{\mathrm{bb}}_{k+1} = f_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x^{\mathrm{bb}}_k, r_k, u^{\mathrm{ms}}_k\big), \quad \hat{y}_k = r_k + h_{\mathrm{bb}}\big(\theta_{\mathrm{bb}}, x^{\mathrm{bb}}_k\big), \tag{52}$$

where $x^{\mathrm{bb}}_k \in \mathbb{R}^{n_x}$ and an encoder estimates x^{bb} at the window start from past inputs and servo errors, as in [16]. Its settings are the black-box rows of Table XII.

E. Hyperparameters

Table XII lists the settings of every grey-box run and of the black-box search. The basis of a value is derived (computed in Section VI or Appendix D), inherited (from earlier work), tuned (on validation data), heuristic (fixed without a measurement) or compute-limited. Every model is trained once with one seed, so differences between models smaller than the variation between seeds are not resolved. [**TODO:** Missing: a stated reason for the rows marked heuristic without one in `docs/decisions.md` (window stride, batch, RK4 steps, the PS rules). Candidate: keep them as heuristic, with D-234 as the source of the PS rules (prototype runs).] [**TODO:** Missing: the black-box rows are the search grid of `thesis_r8_grid.conf`; the selected configuration is not known yet. Candidate: add a row with the selected ~~n_x, width and~~ rate once R8 has run.]

F. OBC and experiment details

- Derive the stacked projector, rank-deficient pseudoinverse case, epoch refresh, and additional-state zero extension
- Derive $J^{+}\Delta^{*}$ and list the measured singular-direction contributions to the predicted parameter bias
- List dataset identifiers, split membership, hyperparameters, seed, and exact metric definitions for every reported arm
- Figure (existing, optional): place `controller-chain-v1` here only if the controller reconstruction cannot be audited adequately in prose and equations

[**TODO:** Can code and data tables be cited through a repository or DOI instead of spending paper space on full reproducibility metadata?]